UNIVERSITATEA DE ȘTIINȚE ȘI TEHNOLOGIE POLITEHNICA BUCUREȘTI

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Programul de masterat: Ingineria Informației și Sisteme de Calcul

Sistem E-Medicine pentru Analizarea și Monitorizarea Datelor Epidemiologice bazat pe Modelul Matematic SIDARTH

LUCRARE DE DISERTAȚIE

Coordonator științific:

Prof. Radu Rădescu

Absolvent:

Geană Tudor-Bogdan

2026

Rezumat

Această lucrare tratează proiectarea unui sistem e-medicine pentru analiza datelor epidemiologice prin modele compartimentale, cu modelul SIDARTH folosit ca direcție principală de explorare. Spre deosebire de SIR, care reduce dinamica la trei stări agregate, SIDARTH separă infecțiile nedepistate, cazurile diagnosticate, formele simptomatice, formele severe și vindecările. Această granularitate face modelul mai potrivit pentru discutarea efectelor testării, diagnosticării și severității clinice asupra evoluției unei epidemii.

SIR rămâne prezent în lucrare ca model secundar, folosit pentru comparații și pentru verificarea comportamentului numeric al implementărilor. Sistemul construit separă explicit stocarea datelor, expunerea API-ului și execuția simulărilor. Datele epidemiologice sunt persistate într-o schemă relațională compatibilă PostgreSQL, rulată peste YugabyteDB, iar simulările sunt executate asincron de runner-e specializate, coordonate prin Redis.

Componenta web, alcătuită dintr-un API Go și o interfață Vue.js, oferă fluxurile necesare încărcării seturilor de date, lansării simulărilor, urmăririi statusului și vizualizării traiectoriilor compartimentale. Accentul lucrării cade pe legătura dintre modelul matematic, implementarea numerică și arhitectura software care face posibilă evaluarea reproductibilă a rezultatelor pentru modele cu grade diferite de complexitate.

Cuvinte cheie: model SIDARTH, model SIR, epidemiologie computațională, e-medicine, calcul comparativ, YugabyteDB, Kubernetes, Go, Vue.js, benchmark de performanță.

Cuprins

Rezumat **i**

Listă de figuri **vi**

Listă de tabele **vii**

Lista algoritmilor **viii**

1 Introducere **1**

1.1 Context și motivație 1
1.2 Obiectivele lucrării 2
1.3 Contribuții originale 2
1.4 Structura lucrării 2

2 Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase **4**

2.1 Fundamente de epidemiologie 4
2.1.1 Definiții și concepte fundamentale 4
2.1.2 Parametrii fundamentali ai transmiterii 5
2.2 Modele compartimentale în epidemiologie 5
2.2.1 Cadrul general 5
2.2.2 Modelul SI 6
2.2.3 Modelul SIS 6
2.2.4 Modelul SEIR 7
2.2.5 Modelul SIRS 7
2.3 Modelul SIDARTH 7
2.4 Modelul SIR ca reper secundar 9
2.4.1 Ipoteze și derivarea ecuațiilor din principii prime 9
2.4.2 Proprietăți matematice fundamentale 10
2.4.3 Analiza echilibrelor și stabilitate 11
2.4.4 Numărul de reproducere de bază $\mathcal{R}_0$ 12
2.4.5 Pragul de imunizare în turmă (herd immunity) 14

2.4.6 Ecuația dimensiunii finale a epidemiei . . . 14
2.4.7 Extensii ale modelului SIR . . . 15
2.5 Metode Numerice pentru Ecuații Diferențiale Ordinare . . . 16
2.5.1 Clasificarea metodelor numerice . . . 16
2.5.2 Metoda Euler explicită . . . 16
2.5.3 Metoda Runge-Kutta de ordinul 4 (RK4) . . . 17
2.5.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince) . . . 18
2.5.5 Stabilitate numerică și probleme stiff . . . 19
2.6 Date Epidemiologice — Surse și Preprocesare . . . 20
2.6.1 Surse de date epidemiologice . . . 20
2.6.2 Structura datelor și preprocesare . . . 20
2.6.3 Estimarea parametrilor din date reale . . . 20

3 Tehnologii și Instrumente Utilizate 22
3.1 Limbaje de programare pentru calcul științific . . . 22
3.1.1 R . . . 22
3.1.2 Julia . . . 23
3.1.3 Python . . . 24
3.1.4 Analiză comparativă a limbajelor . . . 25
3.2 Baze de date distribuite — YugabyteDB . . . 25
3.2.1 Concepte de baze de date distribuite . . . 25
3.2.2 Arhitectura YugabyteDB . . . 26
3.2.3 Sharding și replicare . . . 26
3.3 Orchestrarea containerelor — Kubernetes . . . 26
3.3.1 Containere și nevoia de orchestrare . . . 26
3.3.2 Arhitectura Kubernetes . . . 27
3.3.3 Resurse Kubernetes relevante . . . 27
3.4 Backend API — Go . . . 27
3.4.1 Caracteristici și filosofia limbajului Go . . . 27
3.4.2 Ecosistem pentru API-uri REST . . . 28
3.5 Frontend — Vue.js . . . 28
3.5.1 Arhitectura Vue 3 și Composition API . . . 28
3.5.2 Ecosistem și biblioteci utilizate . . . 28

4 Arhitectura Sistemului E-Medicine 29
4.1 Vedere de ansamblu . . . 29
4.2 Cerințe funcționale și non-funcționale . . . 30
4.2.1 Cerințe funcționale . . . 30
4.2.2 Cerințe non-funcționale . . . 31
4.3 Arhitectura bazei de date distribuite . . . 32

4.3.1 Schema relațională 32
4.3.2 Modelul datelor epidemiologice 33
4.3.3 Strategia de partiționare și replicare în YugabyteDB 33
4.4 Arhitectura API-ului backend Go 35
4.4.1 Structura modulară a proiectului 35
4.4.2 Mecanismul de autentificare JWT 36
4.4.3 Diagrama endpointurilor REST 36
4.4.4 Integrarea cu runner-ele de algoritmi 37
4.5 Arhitectura frontend Vue.js 37
4.5.1 Structura componentelor 37
4.5.2 Fluxul de date și state management 38
4.6 Infrastructura Kubernetes 39
4.6.1 Topologia cluster-ului 39
4.6.2 Deployment YugabyteDB pe Kubernetes 39
4.6.3 Deployment API Go și frontend Vue.js 40

5 Implementarea modelelor SIDARTH și SIR 42
5.1 Datele de intrare și inițializarea compartimentelor 42
5.2 Contractul comun de simulare 43
5.3 Implementarea modelului SIDARTH 44
5.4 SIR ca model secundar 45
5.5 Persistența rezultatelor și exportul 45
5.6 Metrici de performanță 46

6 Evaluarea Performanței și Scalabilității 47
6.1 Principiul evaluării 47
6.2 Metrici colectate 47
6.3 Protocol experimental 48
6.4 Scalabilitate și complexitatea modelului 48
6.5 Compararea implementărilor R, Julia și Python 48
6.6 Acuratețe numerică 49
6.7 Interpretarea rezultatelor 49

7 Aplicația Web E-Medicine 50
7.1 Rolul aplicației 50
7.2 Backend Go 50
7.3 Gestionarea dataseturilor 51
7.4 Execuția simulărilor 51
7.5 Frontend Vue.js 51
7.6 Validare funcțională 52

8 Concluzii și Direcții de Dezvoltare Viitoare **53**
8.1 Sinteza lucrării 53
8.2 Concluzii privind modelele 53
8.3 Concluzii privind implementarea 54
8.4 Limitări 54
8.5 Direcții viitoare 54

Bibliografie **56**

A Cod Sursă — Implementarea R **58**

B Cod Sursă — Implementarea Julia **65**

C Cod Sursă — Implementarea Python **71**

D Schema Bazei de Date **78**

E Configurații Kubernetes **81**

F Documentația API REST **98**

Listă de figuri

2.1 Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare. . . . 10

4.1 Arhitectura globală a sistemului e-medicine. Săgețile indică direcția de flux a datelor: browserul accesează aplicația prin Ingress, care rutează cererile către SPA sau API; API-ul publică job-uri în Redis și le consumă runner-ele dedicate; atât API-ul, cât și runner-ele accesează același cluster YugabyteDB. 30

4.2 Diagrama entitate-relație a bazei de date. PK-urile sunt marcate cu aldine și subliniere; FK-urile sunt marcate cu *FK*; raportul de cardinalitate 1 : N este indicat pe fiecare relație. 32

4.3 Topologia cluster-ului Kubernetes. Ingressul este expus ca LoadBalancer Service în fața clusterului. Worker Node 1 găzduiește frontend-ul, primul runner și prima instanță YugabyteDB tserver; Worker Node 2 găzduiește celelalte două runner-e și doi tserveri. 40

Listă de tabele

3.1 Comparație a limbajelor de programare utilizate 25

4.1 Structura minimă a fișierului CSV de date epidemiologice. 33
4.2 Endpointurile REST ale API-ului backend. 36

5.1 Maparea datelor observabile în condiții inițiale pentru modelele suportate. 43

6.1 Metricile colectate pentru fiecare rulare. 47

E.1 Manifestele Kubernetes incluse în repository. 82

Lista algoritmilor

2.1 Metoda Euler pentru sistemul SIR 17
2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR 18

Capitolul 1

Introducere

1.1 Context și motivație

Bolile infecțioase rămân una dintre formele prin care vulnerabilitatea unui sistem sanitar devine vizibilă la scară socială. Pandemia de gripă din 1918, epidemia de SARS din 2003 și pandemia de COVID-19 declanșată în 2019 arată că propagarea unui agent patogen nu poate fi înțeleasă doar ca fenomen clinic, ci și ca proces populațional, dependent de contacte, mobilitate, raportare și capacitatea de răspuns instituțional [1, 2].

Modelele matematice compartimentale oferă un limbaj riguros pentru descrierea acestor procese. Ele reduc populația la stări epidemiologice relevante și descriu tranzițiile dintre stări prin ecuații diferențiale. Modelul SIR (Susceptibili–Infectați–Recuperați), introdus de Kermack și McKendrick în 1927 [3], rămâne un reper important deoarece permite o analiză matematică transparentă. Totuși, pentru epidemii în care testarea, diagnosticul și severitatea clinică modifică puternic raportarea, un model cu trei compartimente este prea agregat. Din acest motiv, lucrarea folosește SIDARTH ca model principal de explorare, păstrând SIR ca model secundar pentru comparații și verificări.

În practica actuală, problema nu se oprește la scrierea ecuațiilor. Datele provin din surse eterogene, sunt actualizate frecvent și conțin corecții, goluri sau inconsistențe administrative. În același timp, implementarea numerică poate fi realizată în limbaje diferite, fiecare cu avantaje și costuri proprii. Alegerea dintre R, Julia și Python nu ține doar de sintaxă, ci influențează timpul de execuție, consumul de memorie, modul de integrare cu infrastructura și reproductibilitatea analizei.

Pornind de la aceste observații, lucrarea propune un sistem e-medicine în care modelul SIDARTH este integrat într-un flux complet: ingestie de date epidemiologice, execuție asincronă a simulărilor, stocare a rezultatelor și vizualizare în interfață web. SIR și variațiunile apropiate rămân utile ca repere mai simple, prin care se poate observa ce se pierde sau ce se câștigă atunci când modelul introduce compartimente suplimentare.

1.2 Obiectivele lucrării

Obiectivul principal este proiectarea unui sistem care leagă explicit modelarea matematică de implementarea software. Modelul SIDARTH este tratat ca model de lucru principal, deoarece separă cazurile nedepistate, diagnosticate, simptomatice, severe și vindecate. Modelul SIR este analizat în continuare, dar rolul său devine acela de reper teoretic și computațional, util pentru înțelegerea proprietăților de bază ale modelelor compartimentale.

Al doilea obiectiv este implementarea aceluiași contract de simulare în R, Julia și Python, folosind bibliotecile uzuale ale fiecărui ecosistem pentru rezolvarea ecuațiilor diferențiale. Comparația urmărește atât compatibilitatea rezultatelor, cât și costurile practice ale execuției: timp wall-clock, timp CPU, memorie și integrare cu baza de date.

În paralel, lucrarea construiește infrastructura necesară rulării acestor experimente într-un cadru accesibil prin web. API-ul Go, baza de date YugabyteDB, coada Redis și interfața Vue.js sunt tratate ca părți ale aceluiași instrument de cercetare, nu ca module izolate.

1.3 Contribuții originale

Contribuția lucrării constă în combinarea unei analize matematice clasice cu un sistem software funcțional, capabil să ruleze modele compartimentale cu structuri de stare diferite în aceleași medii de execuție. Partea matematică fixează ipotezele și limitele modelelor, partea de implementare arată cum se schimbă costul numeric între limbaje, iar partea de arhitectură oferă un cadru în care datele, simulările și benchmarkurile pot fi gestionate unitar.

O a doua contribuție este metodologia de colectare a metricilor. Fiecare runner marchează explicit stările unei simulări, scrie rezultatele în aceeași schemă de bază de date și salvează metrici comparabile. Această disciplină de integrare este necesară pentru a evita comparațiile informale între limbaje, care pot fi ușor influențate de diferențe de I/O, inițializare sau serializare.

În fine, sistemul web rezultat permite folosirea modelelor într-un flux apropiat de un scenariu real de analiză: încărcarea unui set de date, alegerea modelului și a parametrilor, execuția asincronă, vizualizarea traiectoriilor și compararea rulărilor.

1.4 Structura lucrării

Lucrarea este organizată în opt capitole.

Capitolul 2 (*Epidemiologie Computațională și Modelarea Matematică*) prezintă fundamentele epidemiologiei matematice. Sunt introduse modelele compartimentale

(SI, SIS, SIR, SEIR, SIRS), apoi este prezentat modelul SIDARTH ca model principal de explorare. Modelul SIR este păstrat pentru analiza matematică de bază, deoarece permite discutarea clară a pozitivității, conservării populației și pragului epidemic $\mathcal{R}_0$. Capitolul include și o trecere în revistă a metodelor numerice utilizate pentru rezolvarea sistemelor de ecuații diferențiale ordinare.

Capitolul 3 (*Tehnologii și Instrumente Utilizate*) explică de ce au fost alese limbajele R, Julia și Python, baza de date YugabyteDB, orchestrarea containerelor prin Kubernetes, backend-ul Go și frontend-ul Vue.js.

Capitolul 4 (*Arhitectura Sistemului E-Medicine*) descrie arhitectura globală a sistemului, cerințele funcționale și non-funcționale, schema bazei de date distribuite, arhitectura API-ului backend și structura componentelor frontend.

Capitolul 5 (*Implementarea Modelelor SIDARTH și SIR*) detaliază implementările practice ale modelelor în fiecare limbaj de programare, inclusiv seturile de date utilizate, parametrii, schema generică de rezultate și interfața de integrare cu API-ul Go.

Capitolul 6 (*Evaluarea Performanței și Scalabilității*) prezintă metodologia de benchmarking, configurațiile hardware utilizate și rezultatele comparative ale evaluărilor de performanță.

Capitolul 7 (*Aplicația Web E-Medicine*) detaliază implementarea aplicației web, documentând atât backend-ul Go, cât și frontend-ul Vue.js, cu descrierea funcționalităților și a fluxurilor de utilizare.

Capitolul 8 (*Concluzii și Direcții Viitoare*) sintetizează contribuțiile lucrării, discută limitele sistemului și identifică direcții de dezvoltare ulterioară.

Lucrarea se încheie cu **Bibliografie** și **Anexe** care conțin codul sursă complet al implementărilor, schema bazei de date, configurațiile Kubernetes și documentația API-ului REST.

Capitolul 2

Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase

2.1 Fundamente de epidemiologie

2.1.1 Definiții și concepte fundamentale

Epidemiologia este știința care studiază distribuția, determinanții și controlul stărilor sau evenimentelor legate de sănătate în populații definite [1]. În sens mai specific, *epidemiologia infecțioasă* se ocupă cu dinamica transmiterii bolilor cauzate de agenți patogeni biologici (virusuri, bacterii, paraziți) în populații umane sau animale.

Definiție 2.1 (Epidemic, pandemic, endemic)**.** O boală infecțioasă se numește **endemică** dacă persistă la nivel constant într-o regiune geografică definită. Se numește **epidemie** dacă numărul de cazuri depășește clar nivelul endemic așteptat, și **pandemie** dacă afectează mai multe continente sau o mare parte a globului.

Definiție 2.2 (Incidență și prevalență)**. Incidența** reprezintă numărul de cazuri noi ale unei boli apărute într-o populație de risc, într-un interval de timp specificat. Se exprimă ca rată:

$$\text{Incidență} = \frac{\text{cazuri noi în intervalul } [t_1, t_2]}{\text{populație la risc} \times (t_2 - t_1)}. \tag{2.1}$$

Prevalența reprezintă proporția indivizilor afectați la un moment dat:

$$\text{Prevalență} = \frac{\text{cazuri existente la momentul } t}{\text{populație totală la momentul } t}. \tag{2.2}$$

2.1.2 Parametrii fundamentali ai transmiterii

Definiție 2.3 (Intervalul serial și intervalul de generație)**. Intervalul serial** este durata medie dintre apariția simptomelor la cazul primar și la cazul secundar generat de acesta. **Intervalul de generație** este durata medie dintre infecția cazului primar și infecția unui caz secundar.

Definiție 2.4 (Perioada de incubație)**. Perioada de incubație** este intervalul de timp dintre momentul infectării și apariția primelor simptome. Este distinctă de *perioada de latență*, care reprezintă intervalul dintre infecție și debutul infecțiozității.

Definiție 2.5 (Numărul de reproducere de bază $\mathcal{R}_0$)**.** Numărul de reproducere de bază $\mathcal{R}_0$ este definit ca numărul mediu de infecții secundare produse de un singur individ infectat introdus într-o populație complet susceptibilă, pe durata perioadei sale infecțioase [4]. Este cel mai important parametru al epidemiologiei matematice, deoarece determină potențialul epidemic al unui agent patogen.

O consecință imediată a Teoremă 2.5 este existența unui prag epidemic. Dacă $\mathcal{R}_0 < 1$, fiecare caz infectat produce în medie mai puțin de un caz nou, iar epidemia se stinge spontan. Dacă $\mathcal{R}_0 > 1$, numărul de cazuri crește în faza inițială, de regulă cu profil aproximativ exponențial. Cazul $\mathcal{R}_0 = 1$ separă cele două regimuri și corespunde unei stări de echilibru instabil.

Numărul de reproducere *efectiv* (sau în timp real) $\mathcal{R}(t)$ generalizează $\mathcal{R}_0$ pentru o populație parțial imunizată:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N}, \tag{2.3}$$

unde $S(t)$ este numărul de indivizi susceptibili la momentul t și N este mărimea populației totale.

2.2 Modele compartimentale în epidemiologie

2.2.1 Cadrul general

Modelele compartimentale reprezintă clasa fundamentală de modele matematice în epidemiologie [2]. Principiul lor constă în împărțirea populației în *compartimente* care descriu stări epidemiologice distincte, iar tranziția dintre compartimente este guvernată de un sistem de ecuații diferențiale ordinare (EDO).

Fie N mărimea totală a populației (presupusă constantă). Fiecare compartiment $X_i(t)$ reprezintă numărul de indivizi în starea i la momentul t, cu $\sum_i X_i(t) = N$. Dinamica tranzițiilor este descrisă de sistemul:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t, \boldsymbol{\theta}), \quad \mathbf{x}(0) = \mathbf{x}_0, \tag{2.4}$$

unde $\mathbf{x} = (X_1, \ldots, X_n)^\intercal$, $\boldsymbol{\theta}$ este vectorul parametrilor și $\mathbf{f}$ descrie fluxurile nete dintre compartimente.

2.2.2 Modelul SI

Cel mai simplu model compartimențal este modelul **SI** (Susceptibili-Infectați), care presupune că infecția este permanentă (nu există recuperare):

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N}, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N}. \end{cases} \tag{2.5}$$

Cu condiția $S + I = N$, sistemul (2.5) se reduce la o ecuație logistică în I, cu soluția analitică:

$$I(t) = \frac{N}{1 + \left(\frac{N - I_0}{I_0}\right) e^{-\beta t}}, \tag{2.6}$$

unde $I_0 = I(0)$. Toți indivizii sunt infectați pe termen lung ($I(\infty) = N$).

2.2.3 Modelul SIS

Modelul **SIS** (Susceptibili-Infectați-Susceptibili) extinde modelul SI prin includerea recuperării fără imunitate, permițând re-infectarea:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \gamma I, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.7}$$

Parametrul $\gamma > 0$ reprezintă rata de recuperare ($1/\gamma$ este durata medie a perioadei infecțioase). Pentru modelul SIS, $\mathcal{R}_0 = \beta/\gamma$. Dacă $\mathcal{R}_0 \leq 1$, echilibrul fără boală $I^* = 0$ este stabil global; dacă $\mathcal{R}_0 > 1$, sistemul admite un echilibru endemic stabil $I^* = N(1 - 1/\mathcal{R}_0) > 0$.

2.2.4 Modelul SEIR

Modelul **SEIR** introduce compartimentul E (Expuși / în perioadă de latență), care ține cont de faptul că indivizii infectați nu sunt imediat infecțioși:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N}, \\ \frac{dE}{dt} = \frac{\beta SI}{N} - \sigma E, \\ \frac{dI}{dt} = \sigma E - \gamma I, \\ \frac{dR}{dt} = \gamma I, \end{cases} \tag{2.8}$$

unde $\sigma = 1/T_E$ este rata de tranziție din starea expusă în starea infecțioasă (T_E = durata medie de incubație). Numărul de reproducere de bază pentru modelul SEIR este $\mathcal{R}_0 = \beta/\gamma$, identic cu cel al modelului SIR.

2.2.5 Modelul SIRS

Modelul **SIRS** generalizează modelul SIR prin imunitate temporară (indivizii recuperați pot deveni din nou susceptibili după un interval de timp):

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \xi R, \\ \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} = \gamma I - \xi R, \end{cases} \tag{2.9}$$

unde $\xi = 1/T_R$ este rata de pierdere a imunității (T_R = durata medie a imunității). Dacă $\mathcal{R}_0 > 1$, modelul SIRS admite un echilibru endemic stabil.

2.3 Modelul SIDARTH

Modelul SIDARTH folosit în această lucrare este o variantă cu șapte compartimente a familiei SIDARTHE propuse pentru analiza epidemiei COVID-19 [5]. Alegerea lui ca model principal este motivată de faptul că separă două dimensiuni care lipsesc din SIR: statutul de diagnosticare și severitatea clinică. Într-un sistem de raportare real, persoanele infectate dar nedepistate, persoanele diagnosticate, cazurile simptomatice și cazurile severe nu au aceeași contribuție la transmitere și nu sunt observate cu aceeași probabilitate.

Compartimentele modelului sunt S (susceptibili), I (infectați nedepistați), D (diagnosticați asimptomatici), A (simptomatici nedepistați), R (simptomatici diagnosticați),

T (cazuri severe) și H (vindecați). În forma implementată, compartimentul E din modelul SIDARTHE original, asociat deceselor, este omis. Această alegere reduce numărul de parametri și concentrează analiza pe tranzițiile relevante pentru diagnostic, severitate și vindecare; decesele pot fi adăugate ulterior printr-un termen de ieșire din T.

$$\frac{dS}{dt} = -S \cdot \frac{\alpha I + \beta D + \gamma A + \delta R}{N}, \tag{2.10}$$

$$\frac{dI}{dt} = S \cdot \frac{\alpha I + \beta D + \gamma A + \delta R}{N} - (\varepsilon + \zeta + \lambda)I, \tag{2.11}$$

$$\frac{dD}{dt} = \varepsilon I - (\eta + \rho)D, \tag{2.12}$$

$$\frac{dA}{dt} = \zeta I - (\theta + \mu + \kappa)A, \tag{2.13}$$

$$\frac{dR}{dt} = \eta D + \theta A - (\nu + \xi)R, \tag{2.14}$$

$$\frac{dT}{dt} = \mu A + \nu R - \sigma T, \tag{2.15}$$

$$\frac{dH}{dt} = \lambda I + \rho D + \kappa A + \xi R + \sigma T. \tag{2.16}$$

Parametrii α, β, γ și δ controlează contribuția la transmitere a compartimentelor infecțioase. Parametrii ε, ζ și λ descriu ieșirile din starea infectată nedepistată: diagnosticare, apariția simptomelor și vindecare fără diagnostic. Restul parametrilor descriu tranzițiile către forme diagnosticate, severe sau vindecate. Spre deosebire de SIR, unde aproape toată interpretarea se concentrează în raportul β/γ, SIDARTH distribuie dinamica pe un număr mai mare de rate; de aceea, analiza numerică și sensibilitatea la parametri devin mai importante decât obținerea unei formule scalare simple.

Conservarea populației se verifică prin însumarea ecuațiilor din (2.16). Toți termenii interni se anulează, iar rezultatul este

$$\frac{d}{dt}(S + I + D + A + R + T + H) = 0. \tag{2.17}$$

Prin urmare, pentru condiții inițiale nenegative, suma compartimentelor rămâne egală cu populația totală N. Pozitivitatea soluției se obține prin același argument folosit la modelele compartimentale clasice: pe frontiera fiecărui compartiment, derivata nu poate împinge soluția în regiunea negativă, deoarece termenii de ieșire dispar când variabila respectivă este zero.

2.4 Modelul SIR ca reper secundar

2.4.1 Ipoteze și derivarea ecuațiilor din principii prime

Modelul SIR, formulat de Kermack și McKendrick [3], se bazează pe o serie de ipoteze simplificatoare care trebuie înțelese explicit:

1. **Populație închisă și omogenă:** Nu există nașteri, decese naturale sau migrații; $N = \text{const}$. Fiecare individ are aceeași probabilitate de a intra în contact cu orice alt individ (amestecare omogenă).

2. **Transmitere prin acțiune de masă:** Rata de contact eficient (producătoare de infecție) este proporțională cu produsul $S \cdot I$.

3. **Imunitate permanentă:** Un individ recuperat nu poate fi re-infectat; compartimentul R este absorbant.

4. **Durata infecțioasă exponențial distribuită:** Rata de recuperare γ este constantă, iar durata medie a perioadei infecțioase este $1/\gamma$.

5. **Fără mortalitate indusă de boală:** Sau, echivalent, mortalitatea specifică bolii este neglijabilă față de dinamica transmiterii.

Derivarea termenului de infecție. Considerăm că în unitatea de timp, un individ susceptibil are în medie c contacte cu alți indivizi aleși uniform din populație. Probabilitatea ca un contact arbitrar să fie cu un individ infectat este I/N, iar probabilitatea de transmitere la contact eficient este p. Astfel, rata de infecție a unui susceptibil este $c \cdot p \cdot I/N = \beta I/N$, unde $\beta = cp$ este parametrul de transmitere. Fluxul total de noi infecții pe unitate de timp este $(\beta I/N) \cdot S$.

Sistemul de ecuații diferențiale ordinare al modelului SIR este:

$$\boxed{\begin{aligned} \frac{dS}{dt} &= -\frac{\beta SI}{N}, \\ \frac{dI}{dt} &= \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, \end{aligned}} \tag{2.18}$$

cu condiții inițiale $S(0) = S_0 > 0$, $I(0) = I_0 > 0$, $R(0) = R_0 \geq 0$ și $S_0 + I_0 + R_0 = N$.

Diagrama de flux a compartimentelor este prezentată în Fig. 2.1.

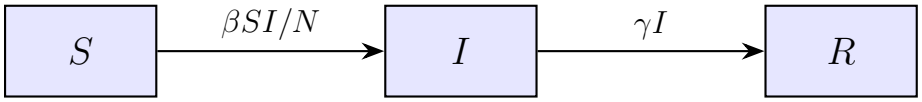


Figura 2.1: Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare.

2.4.2 Proprietăți matematice fundamentale

Înainte de a analiza comportamentul calitativ al soluțiilor, stabilim proprietățile matematice esențiale: existența și unicitatea soluției, conservarea populației totale și invarianța domeniului pozitiv.

Teoremă 2.6 (Existență și unicitate globală)**.** *Fie $\beta, \gamma > 0$, $N > 0$ și $(S_0, I_0, R_0) \in \mathbb{R}^3$ cu $S_0, I_0 > 0$, $R_0 \geq 0$. Atunci sistemul* (2.18) *admite o unică soluție globală $(S, I, R) \in C^1([0, \infty); \mathbb{R}^3)$.*

Demonstrație. Definim $\mathbf{x} = (S, I, R)^\intercal$ și $\mathbf{f}(\mathbf{x}) = (-\beta SI/N,\ \beta SI/N - \gamma I,\ \gamma I)^\intercal$. Funcția $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^3$ este de clasă C^∞ (polinomul de gradul doi în S și I este neted), în particular continuu diferențiabilă. Prin *teorema Picard-Lindelöf* (Cauchy-Lipschitz), există o soluție locală unică pe un interval $[0, T_{\max})$.

Existența *globală* ($T_{\max} = \infty$) se stabilește prin mărginirea soluției. Deoarece $S(t), I(t), R(t) \geq 0$ (Teorema 2.8) și $S + I + R = N$ (Teorema 2.7), rezultă că $\|\mathbf{x}(t)\| \leq N\sqrt{3}$ pentru orice $t \in [0, T_{\max})$. Soluția mărginită nu poate prezenta blow-up în timp finit, deci $T_{\max} = \infty$. □

Teoremă 2.7 (Conservarea populației totale)**.** *Pentru orice soluție a sistemului* (2.18)*,*

$$S(t) + I(t) + R(t) = N \quad \forall\, t \geq 0. \tag{2.19}$$

Demonstrație. Calculăm derivata sumei:

$$\begin{aligned} \frac{d}{dt}(S + I + R) &= \frac{dS}{dt} + \frac{dI}{dt} + \frac{dR}{dt} \\ &= -\frac{\beta SI}{N} + \left(\frac{\beta SI}{N} - \gamma I\right) + \gamma I \\ &= 0. \end{aligned}$$

Prin integrare, $S(t) + I(t) + R(t) = S_0 + I_0 + R_0 = N$. □

Teoremă 2.8 (Pozitivitate și mărginire)**.** *Dacă $S_0 > 0$, $I_0 > 0$, $R_0 \geq 0$, atunci:*

$$S(t) > 0, \quad I(t) > 0, \quad R(t) \geq 0, \quad \forall\, t > 0. \tag{2.20}$$

În particular, $S(t), I(t), R(t) \in [0, N]$ pentru orice $t \geq 0$.

Demonstrație. **(a) Pozitivitatea lui S:** Din ecuația $dS/dt = -(\beta I/N)S$, aceasta este o ecuație diferențială liniară omogenă în S cu coeficient $-\beta I(t)/N$. Soluția este:

$$S(t) = S_0 \cdot \exp\left(-\frac{\beta}{N}\int_0^t I(s)\,ds\right). \tag{2.21}$$

Deoarece integrala este finită și $S_0 > 0$, rezultă $S(t) > 0$ pentru orice $t \geq 0$ (presupunând $I(s) \geq 0$, demonstrat mai jos).

(b) Pozitivitatea lui I: Rescrierea ecuației a doua ca $dI/dt = (\beta S(t)/N - \gamma)\, I$ arată că aceasta este liniară în I:

$$I(t) = I_0 \cdot \exp\left(\int_0^t \left[\frac{\beta S(\tau)}{N} - \gamma\right] d\tau\right) > 0, \tag{2.22}$$

deoarece $I_0 > 0$ și exponențiala este strict pozitivă.

(c) Non-negativitatea lui R: Din $dR/dt = \gamma I(t) > 0$ și $R_0 \geq 0$, funcția R este strict crescătoare, deci $R(t) > R_0 \geq 0$.

Mărginirea urmează din Teorema 2.7: $S(t) \leq N$, $I(t) \leq N$, $R(t) \leq N$. □

Observație 2.9. Teorema 2.8 implică că domeniul pozitiv

$$\Omega = \{(S, I, R) \in \mathbb{R}^3_{\geq 0} : S + I + R = N\} \tag{2.23}$$

este *invariant pozitiv* față de fluxul sistemului (2.18). Prin urmare, este suficient să studiem comportamentul soluțiilor pe Ω, care este un simplex în $\mathbb{R}^3$.

Teoremă 2.10 (Monotonia compartimentelor)**.** *De-a lungul oricărei soluții a sistemului* (2.18)*:*

(i) $S(t)$ *este strict descrescătoare;*

(ii) $R(t)$ *este strict crescătoare;*

(iii) $I(t)$ *atinge un singur maxim în* $t^* = \inf\{t > 0 : S(t) = \gamma N/\beta\}$ *(dacă* $S_0 > \gamma N/\beta$*); altfel este descrescătoare.*

Demonstrație. (i) $dS/dt = -\beta SI/N < 0$ deoarece $S > 0$, $I > 0$. *(ii)* $dR/dt = \gamma I > 0$. *(iii)* $dI/dt = (\beta S/N - \gamma)I > 0 \Leftrightarrow S > \gamma N/\beta$. Deoarece S este strict descrescătoare, există cel mult un moment t^* când $S(t^*) = \gamma N/\beta$, care este maximul lui I. □

2.4.3 Analiza echilibrelor și stabilitate

Punctele de echilibru ale sistemului (2.18) sunt obținute din condiția $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Ecuațiile conduc la:

$$-\frac{\beta S^* I^*}{N} = 0, \quad \frac{\beta S^* I^*}{N} - \gamma I^* = 0, \quad \gamma I^* = 0. \tag{2.24}$$

Din a treia ecuație, $I^* = 0$. Substituind, obținem că orice punct de forma $(S^*, 0, N - S^*)$ cu $S^* \in [0, N]$ este echilibru. Mulțimea echilibrelor este:

$$\mathcal{E} = \{(S^*, 0, N - S^*) : S^* \in [0, N]\} \subset \Omega. \tag{2.25}$$

Observație 2.11. Spre deosebire de modelele SIS și SIRS, modelul SIR nu admite echilibru endemic cu $I^* > 0$. Aceasta reflectă faptul că, pe termen lung, epidemia se stinge întotdeauna (deși poate infecta o fracție mare din populație înainte de stingere).

Analiza stabilității locale. Deoarece $R = N - S - I$, sistemul se reduce la două ecuații (eliminând R):

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N}, \\ \frac{dI}{dt} = \dfrac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.26}$$

Jacobianul sistemului (2.26) evaluat la un echilibru $(S^*, 0)$ este:

$$J(S^*, 0) = \begin{pmatrix} 0 & -\beta S^*/N \\ 0 & \beta S^*/N - \gamma \end{pmatrix}. \tag{2.27}$$

Valorile proprii sunt $\lambda_1 = 0$ și $\lambda_2 = \beta S^*/N - \gamma$.

Teoremă 2.12 (Stabilitatea echilibrului fără boală)**.** *Echilibrul fără boală* $(S^*, 0)$ *este stabil nelinear dacă* $\beta S^*/N < \gamma$*, adică dacă* $S^*/N < 1/\mathcal{R}_0$*, și este instabil dacă* $\beta S^*/N > \gamma$*.*

Demonstraţie. Valorea proprie $\lambda_2 = \beta S^*/N - \gamma$ determină stabilitatea pe direcția I. Deoarece $\lambda_1 = 0$, analiza liniară este degenerată și trebuie completată cu argumente neliniare. Totuși, din Teorema 2.10, $I(t)$ este descrescătoare dacă $S(t) < \gamma N/\beta$ pentru tot $t > 0$, ceea ce se întâmplă când $S_0 < \gamma N/\beta$, implicând stingerea epidemiei. □

2.4.4 Numărul de reproducere de bază $\mathcal{R}_0$

Definiție prin matricea generației următoare

Metoda *matricei generației următoare* (next-generation matrix), formalizată de Diekmann, Heesterbeek și Metz [4] și extinsă de van den Driessche și Watmough [6], furnizează o procedură sistematică pentru calculul $\mathcal{R}_0$ în modele compartimentale de complexitate arbitrară.

Fie $\mathbf{x}_I$ vectorul compartimentelor infectate. Scriem:

$$\frac{d\mathbf{x}_I}{dt} = \mathcal{F}(\mathbf{x}) - \mathcal{V}(\mathbf{x}), \tag{2.28}$$

unde $\mathcal{F}_i$ reprezintă rata de apariție a noilor infecții în compartimentul i, iar $\mathcal{V}_i$ reprezintă rata netă de ieșire din compartimentul i prin toate celelalte procese.

Matricele Jacobiene evaluate la echilibrul fără boală $\mathbf{x}_0$ sunt:

$$F = \left[\frac{\partial \mathcal{F}_i}{\partial x_j}\right]_{\mathbf{x}_0}, \quad V = \left[\frac{\partial \mathcal{V}_i}{\partial x_j}\right]_{\mathbf{x}_0}. \tag{2.29}$$

Matricea generației următoare este $K = FV^{-1}$, iar:

$$\mathcal{R}_0 = \rho(K), \tag{2.30}$$

unde $\rho(\cdot)$ este raza spectrală (cea mai mare valoare proprie în modul).

Calculul $\mathcal{R}_0$ pentru modelul SIR

Pentru modelul SIR, singurul compartiment infectat este $\{I\}$. La echilibrul fără boală $(S, I, R) = (N, 0, 0)$:

$$\mathcal{F}(I) = \frac{\beta S I}{N} \implies F = \left[\frac{\partial \mathcal{F}}{\partial I}\right]_{S=N} = \frac{\beta N}{N} = \beta. \tag{2.31}$$

$$\mathcal{V}(I) = \gamma I \implies V = \gamma. \tag{2.32}$$

Matricea K este scalarul:

$$K = FV^{-1} = \frac{\beta}{\gamma}. \tag{2.33}$$

Prin urmare:

$$\boxed{\mathcal{R}_0 = \frac{\beta}{\gamma}.} \tag{2.34}$$

Interpretarea biologică este directă: $1/\gamma$ este durata medie a perioadei infecțioase, iar β este numărul mediu de contacte eficiente pe unitate de timp; produsul lor dă numărul de infecții secundare produse pe durata infecțiozității.

Teoremă 2.13 (Pragul epidemic)**.** *Considerăm o populație inițial complet susceptibilă ($S_0 \approx N$). Atunci:*

(i) Dacă $\mathcal{R}_0 > 1$, numărul de infectați crește inițial exponențial și are loc o epidemie ($I(t)$ atinge un maxim $I_{\max} > I_0$).

(ii) Dacă $\mathcal{R}_0 \leq 1$, $I(t)$ este imediat descrescătoare și nu are loc nicio epidemie.

Demonstrație. Din $dI/dt|_{t=0} = I_0(\beta S_0/N - \gamma) \approx I_0(\beta - \gamma)$ (pentru $S_0 \approx N$), concluzia urmează din semnul expresiei $\beta - \gamma$:

$$\left.\frac{dI}{dt}\right|_{t=0} > 0 \iff \beta > \gamma \iff \frac{\beta}{\gamma} > 1 \iff \mathcal{R}_0 > 1. \tag{2.35}$$

□

2.4.5 Pragul de imunizare în turmă (herd immunity)

Definiție 2.14 (Imunitate de grup)**.** Imunitatea de grup (*herd immunity*) se referă la protecția indirectă a persoanelor susceptibile, generată atunci când o fracție suficient de mare din populație este imunizată (fie natural, fie prin vaccinare), astfel încât transmiterea susținută a agentului patogen devine imposibilă.

Epidemia se stinge când numărul de reproducere efectiv devine subunitar:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N} < 1 \iff \frac{S(t)}{N} < \frac{1}{\mathcal{R}_0}. \tag{2.36}$$

Dacă o fracție p din populație este imunizată (cu $S/N = 1 - p$), condiția de blocare a epidemiei devine:

$$(1-p) \cdot \mathcal{R}_0 < 1 \iff p > 1 - \frac{1}{\mathcal{R}_0}. \tag{2.37}$$

Definiție 2.15 (Pragul de imunizare în turmă)**.** Fracția minimă a populației care trebuie imunizată pentru a preveni propagarea epidemiei este:

$$p_c = 1 - \frac{1}{\mathcal{R}_0}. \tag{2.38}$$

Exemplu 2.16. Pentru rujeolă, $\mathcal{R}_0 \approx 12$–18, deci $p_c \approx 92$–94%. Pentru COVID-19 (tulpina originală), $\mathcal{R}_0 \approx 2$–3, deci $p_c \approx 50$–67%. Variantele ulterioare (Delta, Omicron) au $\mathcal{R}_0$ mai mare, crescând corespunzător pragul de imunizare.

2.4.6 Ecuația dimensiunii finale a epidemiei

O caracteristică remarcabilă a modelului SIR este că permite calculul analitic al *dimensiunii finale* a epidemiei — fracția totală din populație care este infectată în cursul epidemiei — fără a rezolva explicit sistemul diferențial.

Teoremă 2.17 (Ecuația dimensiunii finale)**.** *Fie $z = 1 - S(\infty)/N$ fracția din populație infectată cumulativ pe durata epidemiei, presupunând $S_0 = N$ și $R_0 = 0$. Atunci z este unica rădăcină pozitivă a ecuației transcendentale:*

$$z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.39}$$

Demonstrație. Împărțind ecuația pentru S la ecuația pentru R:

$$\frac{dS}{dR} = \frac{-\beta SI/N}{\gamma I} = -\frac{\beta}{\gamma N} S = -\frac{\mathcal{R}_0}{N} S. \tag{2.40}$$

Această ecuație separabilă în S și R se integrează direct:

$$\int_{S_0}^{S(t)} \frac{dS'}{S'} = -\frac{\mathcal{R}_0}{N} \int_0^{R(t)} dR' \implies \ln \frac{S(t)}{S_0} = -\frac{\mathcal{R}_0}{N} R(t). \tag{2.41}$$

Deci $S(t) = S_0 \exp(-\mathcal{R}_0 R(t)/N)$.

La $t \to \infty$, $I(\infty) = 0$ (demonstrat mai jos), deci $R(\infty) = N - S(\infty)$ (pentru $R_0 = 0$). Substituind cu $S_0 = N$:

$$S(\infty) = N e^{-\mathcal{R}_0 (N - S(\infty))/N}. \tag{2.42}$$

Notând $z = 1 - S(\infty)/N$ și împărțind prin N:

$$1 - z = e^{-\mathcal{R}_0 z} \implies z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.43}$$

Că $I(\infty) = 0$: Funcția $R(t)$ este monoton crescătoare și mărginită de N, deci $\lim_{t\to\infty} R(t)$ există. Dacă $I(\infty) = \ell > 0$, atunci $R'(t) = \gamma I(t) \geq \gamma \ell > 0$ pentru t suficient de mare, ceea ce implică $R(t) \to \infty$, contradicție. Deci $I(\infty) = 0$.

Unicitatea rădăcinii pozitive: Fie $g(z) = 1 - e^{-\mathcal{R}_0 z} - z$. Avem $g(0) = 0$, $g'(0) = \mathcal{R}_0 - 1 > 0$ (dacă $\mathcal{R}_0 > 1$), și $g(1) = 1 - e^{-\mathcal{R}_0} - 1 = -e^{-\mathcal{R}_0} < 0$. Deoarece $g''(z) = -\mathcal{R}_0^2 e^{-\mathcal{R}_0 z} < 0$, funcția g este concavă, deci are exact o singură rădăcină în $(0, 1)$. □

Observație 2.18. Ecuația (2.39) este *transcendentală* și nu admite soluție în formă închisă pentru $\mathcal{R}_0 \neq 1$. Se rezolvă numeric, de exemplu prin metoda Newton-Raphson sau prin iterație de punct fix.

Corolar 2.19 (Rata de atac finală)**.** *Dacă $\mathcal{R}_0 > 1$, rata de atac finală z^* satisface $z^* \in (0, 1)$ și crește monoton cu $\mathcal{R}_0$. Limita* $\lim_{\mathcal{R}_0 \to \infty} z^* = 1$.

2.4.7 Extensii ale modelului SIR

Modelul SIR de bază poate fi extins în mai multe direcții. Introducerea nașterilor și mortalității naturale permite studiul echilibrelor endemice pe termen lung. Structura pe vârste poate fi inclusă prin matrice de contact, iar propagarea geografică prin modele cu patch-uri sau ecuații diferențiale parțiale. Pentru populații mici sau pentru fazele incipiente ale unei epidemii, modelele stochastice devin mai potrivite decât formularea deterministă. Intervențiile precum vaccinarea, izolarea și tratamentul se pot introduce ca fluxuri suplimentare între compartimente.

2.5 Metode Numerice pentru Ecuații Diferențiale Ordinare

2.5.1 Clasificarea metodelor numerice

Soluția analitică a sistemului SIR nu există în formă închisă (cu excepția cazurilor degenerate), ceea ce face indispensabilă rezolvarea numerică. O metodă numerică pentru problema cu valori inițiale (PVI):

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(t_0) = \mathbf{y}_0, \tag{2.44}$$

generează o secvență de aproximații $\mathbf{y}_n \approx \mathbf{y}(t_n)$ la punctele $t_n = t_0 + nh$.

Metodele numerice pot fi privite din mai multe unghiuri. Metodele explicite calculează direct $\mathbf{y}_{n+1}$ din valorile anterioare, în timp ce metodele implicite includ necunoscuta și în membrul drept, necesitând rezolvarea unui sistem la fiecare pas. Metodele un-pas folosesc doar starea curentă, pe când metodele multi-pas folosesc și stări anterioare. Ordinul de convergență p determină comportamentul erorii globale, $\|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^p)$. În sfârșit, metodele cu pas adaptiv ajustează h pe baza unei estimări a erorii locale, spre deosebire de schemele cu pas fix.

2.5.2 Metoda Euler explicită

Metoda Euler (1768) este cea mai simplă metodă numerică un-pas, obținută prin truncherea seriei Taylor la ordinul întâi:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \cdot \mathbf{f}(t_n, \mathbf{y}_n). \tag{2.45}$$

Teoremă 2.20 (Eroarea globală a metodei Euler)**.** *Dacă* $\mathbf{f}$ *este Lipschitz în* $\mathbf{y}$ *cu constanta* L *și* $\mathbf{y} \in C^2([t_0, T])$*, eroarea globală a metodei Euler satisface:*

$$\max_{n:t_n \le T} \|\mathbf{y}(t_n) - \mathbf{y}_n\| \le \frac{h}{2L} \max_{t\in[t_0,T]} \|\mathbf{y}''(t)\| \left(e^{L(T-t_0)} - 1\right) = \mathcal{O}(h). \tag{2.46}$$

Eroarea locală de trunchiere (pe un singur pas) este $\mathcal{O}(h^2)$, deci metoda are ordinul local 2 și ordinul global 1.

Aplicată sistemului SIR cu $\mathbf{y}_n = (S_n, I_n, R_n)^\intercal$, schema Euler este:

Stabilitate. Analiza stabilității metodei Euler pe ecuația test $y' = \lambda y$ ($\lambda \in \mathbb{C}$) conduce la factorul de amplificare $z = 1 + h\lambda$. Metoda este stabilă dacă $|1 + h\lambda| \le 1$, ceea ce definește domeniul de stabilitate (disc de rază 1 centrat în -1 în planul $h\lambda$). Această restricție limiteaza pasul de integrare h, în special pentru probleme *stiff*.

Algoritm 2.1 Metoda Euler pentru sistemul SIR

```
Input:  beta, gamma, N, S0, I0, R0, t0, tf, h
Output: Secventa (tn, Sn, In, Rn) pentru tn in [t0, tf]

(S, I, R) <- (S0, I0, R0);  t <- t0
while t < tf do
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    S  <- S + h * dS
    I  <- I + h * dI
    R  <- R + h * dR
    t  <- t + h
    Inregistreaza (t, S, I, R)
end while
```

2.5.3 Metoda Runge-Kutta de ordinul 4 (RK4)

Metodele Runge-Kutta sunt metode un-pas explicite care realizează mai multe evaluări ale funcției $\mathbf{f}$ la fiecare pas, atingând ordine superioare fără a recurge la derivate de ordin înalt ale soluției.

Derivare și formulă

Metoda RK4 clasică (cu 4 stadii) este dată de:

$$\begin{aligned}
\mathbf{k}_1 &= h \cdot \mathbf{f}(t_n, \mathbf{y}_n), \\
\mathbf{k}_2 &= h \cdot \mathbf{f}\left(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_1}{2}\right), \\
\mathbf{k}_3 &= h \cdot \mathbf{f}\left(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_2}{2}\right), \\
\mathbf{k}_4 &= h \cdot \mathbf{f}(t_n + h,\ \mathbf{y}_n + \mathbf{k}_3), \\
\mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right).
\end{aligned} \tag{2.47}$$

Coeficienții metodei sunt reprezentați compact prin *tabloul Butcher*:

$$\begin{array}{c|cccc}
0 & & & & \\
1/2 & 1/2 & & & \\
1/2 & 0 & 1/2 & & \\
1 & 0 & 0 & 1 & \\
\hline
 & 1/6 & 1/3 & 1/3 & 1/6
\end{array} \tag{2.48}$$

Teoremă 2.21 (Ordinul metodei RK4)**.** *Metoda* (2.47) *are ordinul de convergență* $p = 4$*: dacă* $\mathbf{f} \in C^4$ *și* $\mathbf{y} \in C^5([t_0, T])$*, atunci eroarea globală satisface:*

$$\max_n \|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^4). \tag{2.49}$$

Schiță de demonstrație. Eroarea locală de trunchiere τ_n se obține substituind soluția exactă $\mathbf{y}(t_n)$ în schema (2.47) și expandând în serie Taylor. Se poate arăta că primele 4 condiții de ordin (condițiile de arbore Butcher pentru $p = 4$) sunt satisfăcute de coeficienții (2.48), implicând $\tau_n = \mathcal{O}(h^5)$. Eroarea globală este $\mathcal{O}(h^4)$ prin acumularea $\sim 1/h$ de pași. Detalii complete în [7]. □

Algoritm 2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR

```
Input:  f, y0, t0, tf, h
Output: Secventa (tn, yn)

y <- y0;  t <- t0
while t < tf do
    k1 <- h * f(t,        y           )
    k2 <- h * f(t + h/2, y + k1/2     )
    k3 <- h * f(t + h/2, y + k2/2     )
    k4 <- h * f(t + h,   y + k3       )
    y  <- y + (k1 + 2*k2 + 2*k3 + k4) / 6
    t  <- t + h
end while
```

2.5.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince)

Metodele cu pas fix impun utilizatorului alegerea unui h adecvat, care este o sarcină dificilă în practică. Metodele *adaptive* ajustează automat pasul h pe baza unei estimări a erorii locale, asigurând că eroarea rămâne sub o toleranță prestabilită ε.

Metode Runge-Kutta încorporate

O *pereche RK încorporată* de ordine p și $p+1$ calculează doi estimatori pentru $\mathbf{y}_{n+1}$ folosind *aceleași* set de evaluări:

$$\hat{\mathbf{y}}_{n+1} = \mathbf{y}_n + h \sum_{i=1}^{s} b_i \mathbf{k}_i \quad \text{(ordinul } p\text{)}, \tag{2.50}$$

$$\tilde{\mathbf{y}}_{n+1} = \mathbf{y}_n + h \sum_{i=1}^{s} \tilde{b}_i \mathbf{k}_i \quad \text{(ordinul } p+1\text{)}, \tag{2.51}$$

cu $\mathbf{k}_i = \mathbf{f}(t_n + c_i h, \mathbf{y}_n + h\sum_{j<i} a_{ij}\mathbf{k}_j)$. Estimatorul de eroare locală este:

$$\mathbf{e}_{n+1} = \tilde{\mathbf{y}}_{n+1} - \hat{\mathbf{y}}_{n+1} = h\sum_{i=1}^{s}(b_i - \tilde{b}_i)\mathbf{k}_i. \tag{2.52}$$

Metoda Dormand-Prince (DOPRI5 / RK45)

Perechea **Dormand-Prince** [8], implementată ca `ode45` în MATLAB, `RK45` în SciPy (Python), `Tsit5` în Julia și `lsoda`/`ode45` în R (pachetul `deSolve`), utilizează 6 evaluări de funcție și are ordinul principal 5 cu estimator de eroare de ordinul 4.

Tabloul Butcher complet al metodei Dormand-Prince este:

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	−56/15	32/9				
8/9	19372/6561	−25360/2187	64448/6561	−212/729			
1	9017/3168	−355/33	46732/5247	49/176	−5103/18656		
1	35/384	0	500/1113	125/192	−2187/6784	11/84	
$\hat{b}$	35/384	0	500/1113	125/192	−2187/6784	11/84	0
$\tilde{b}$	5179/57600	0	7571/16695	393/640	−92097/339200	187/2100	1/40

(2.53)

Control adaptiv al pasului

Pasul h este actualizat după fiecare pas pe baza normei erorii:

$$\text{err} = \left\| \frac{\mathbf{e}_{n+1}}{\text{atol} + \text{rtol} \cdot |\mathbf{y}_{n+1}|} \right\|_{\infty}, \tag{2.54}$$

unde atol și rtol sunt toleranțe absolute și relative. Pasul următor este calculat prin formula:

$$h_{\text{nou}} = h \cdot \min\!\left(f_{\max},\ \max\!\left(f_{\min},\ S \cdot \text{err}^{-1/(p+1)}\right)\right), \tag{2.55}$$

cu factorul de siguranță $S = 0.9$ și limitele $f_{\min} = 0.1$, $f_{\max} = 10$.

2.5.5 Stabilitate numerică și probleme stiff

Definiție 2.22 (Problemă stiff)**.** O problemă cu valori inițiale (2.44) este numită *stiff* dacă valorile proprii ale Jacobianului $\partial\mathbf{f}/\partial\mathbf{y}$ au componente cu parte reală negativă

de magnitudine mare, impunând pași de integrare h extrem de mici pentru metodele explicite din motive de stabilitate, chiar dacă soluția variază lent.

Aplicație la modelul SIR. Jacobianul sistemului SIR (2.26) este:

$$J(S,I) = \begin{pmatrix} -\beta I/N & -\beta S/N \\ \beta I/N & \beta S/N - \gamma \end{pmatrix}. \tag{2.56}$$

Valorile proprii depind de (S, I). La debutul epidemiei ($S \approx N$, I mic), valorile proprii sunt aproximativ 0 și $\beta - \gamma$, deci problema nu este stiff. Modelul SIR de bază nu este stiff pentru parametri tipici epidemiologici. Totuși, extensiile cu mai multe compartimente și rate foarte diferite pot produce probleme stiff, necesitând metode implicite (Runge-Kutta implicit, BDF).

Observație 2.23. Metodele implicite (de exemplu, metoda Euler implicită, Crank-Nicolson, Radau) au domenii de stabilitate nelimitate și sunt preferate pentru probleme stiff, în ciuda costului computațional superior (rezolvarea unui sistem nonlinear la fiecare pas).

2.6 Date Epidemiologice — Surse și Preprocesare

2.6.1 Surse de date epidemiologice

Implementarea și validarea modelelor epidemiologice computaționale necesită date empirice de calitate. Sursele folosite frecvent includ rapoartele Organizației Mondiale a Sănătății, seriile de supraveghere publicate de ECDC, datasetul COVID-19 al Johns Hopkins University CSSE și colecțiile armonizate de Our World in Data. Această lucrare folosește OWID deoarece oferă serii temporale publice, comparabile între țări și ușor de prelucrat într-un format comun.

2.6.2 Structura datelor și preprocesare

Datele epidemiologice brute conțin frecvent valori lipsă, raportări retrospective, efecte de weekend și schimbări administrative ale unităților geografice. Înainte de utilizarea lor într-un model compartimental, seriile trebuie ordonate temporal, corectate pentru valori imposibile și aduse la o granularitate compatibilă cu pasul de integrare ales. În caz contrar, erorile de raportare pot fi confundate cu variații reale ale dinamicii infecției.

2.6.3 Estimarea parametrilor din date reale

Parametrii β și γ ai modelului SIR pot fi estimați din date observate prin minimizarea erorii pătratice dintre seria raportată și soluția numerică, prin estimare de verosimilitate

maximă cu un model de zgomot adecvat sau prin inferență bayesiană. Alegerea metodei depinde de natura datelor: pentru incidență zilnică, modelele Poisson sau binomial negativ sunt mai potrivite decât o eroare gaussiană simplă, în timp ce inferența bayesiană oferă o descriere explicită a incertitudinii.

Estimatorul $\hat{\gamma}$ poate fi obținut direct dacă durata medie a perioadei infecțioase este disponibilă din studii clinice: $\hat{\gamma} = 1/T_{\text{infecțios}}$. Parametrul β este mai dificil de estimat și depinde de comportamentul social, intervenții și variante ale agentului patogen.

Capitolul 3

Tehnologii și Instrumente Utilizate

3.1 Limbaje de programare pentru calcul științific

Alegerea limbajului de programare pentru implementarea modelelor matematice nu este neutră. Performanța, expresivitatea și ecosistemul de biblioteci influențează atât timpul de dezvoltare, cât și comportamentul numeric al aplicației. În această lucrare, R, Julia și Python sunt tratate ca ecosisteme de calcul științific, nu doar ca sintaxe diferite pentru aceleași formule.

3.1.1 R

Istoric și filosofie

R este un limbaj de programare și mediu software pentru calcul statistic și grafică, descendent direct al limbajului S (Bell Laboratories, 1976). A fost creat de Ross Ihaka și Robert Gentleman la Universitatea din Auckland în 1993, iar în 1997 a fost adoptat ca proiect open-source de *R Development Core Team* [9]. Filosofia R este centrată pe date: vectorii, matricile și cadrele de date (data frames) sunt cetățeni de primă clasă, iar gramatica limbajului este proiectată pentru manipularea și analiza datelor.

Caracteristici tehnice relevante

R este un limbaj *interpretat*, dinamic tipizat, cu suport pentru programare funcțională, procedurală și orientată pe obiecte prin sistemele S3, S4 și R6. Pentru calcul numeric, avantajul său principal vine din vectorizarea nativă: operațiile pe vectori și matrice sunt executate frecvent în cod C sau Fortran, evitând buclele interpretate. În același timp, modelul de memorie cu garbage collector și copiere la scriere poate introduce costuri vizibile atunci când sunt manipulate structuri mari. Posibilitatea de a apela cod nativ prin `.C()`, `.Call()` și `.Fortran()` explică de ce multe pachete R performante ascund implementări compilate în spatele unei interfețe statistice convenabile.

Pachete relevante pentru modele compartimentale

Pentru modelele SIDARTH și SIR, pachetul relevant este `deSolve` [10], care oferă funcția `ode()` și acces la metode consacrate precum `lsoda`, `ode45` și `rk4`. În jurul lui se pot folosi pachete precum `ggplot2` pentru vizualizare, `data.table` pentru prelucrare tabulară și `bench` sau `microbenchmark` pentru măsurarea timpilor de execuție. În implementarea sistemului, runner-ul R folosește `deSolve` pentru integrarea numerică, iar vizualizarea este lăsată aplicației web.

3.1.2 Julia

Istoric și filosofie

Julia a fost creat la MIT de Jeff Bezanson, Alan Edelman, Stefan Karpinski și Viral B. Shah, cu versiunea 1.0 lansată în august 2018 [11]. Motivația proiectului a fost depășirea *problemei celor două limbaje* (two-language problem): cercetătorii scriu prototipuri în R/Python (productivitate ridicată, viteză scăzută) și re-implementează codul critic în C/Fortran (viteză ridicată, productivitate scăzută). Julia vizează ambele obiective simultan.

Compilare Just-In-Time și sistemul de tipuri

Julia utilizează compilare **JIT** (Just-In-Time) bazată pe LLVM, cu o particularitate fundamentală: specializarea pe tipuri concrete. Când o funcție este apelată pentru prima dată cu argumente de tipuri specifice, compilatorul generează cod mașină optimizat pentru acele tipuri exacte. Apelurile ulterioare cu aceleași tipuri rulează direct codul compilat, apropiind performanța de C.

Mecanismul de *multiple dispatch* permite definirea mai multor metode pentru aceeași funcție, în funcție de tipurile argumentelor. Împreună cu sistemul de tipuri parametric, acesta permite compilatorului să genereze cod specializat pentru cazurile folosite efectiv. Macrocomenzile completează acest model prin posibilitatea de a transforma codul la momentul parsării, o tehnică utilizată frecvent în ecosistemul științific Julia.

DifferentialEquations.jl

Ecosistemul **SciML** (Scientific Machine Learning) al Julia conține pachetul `DifferentialEquations.j` [12], un framework open-source matur pentru rezolvarea ecuațiilor diferențiale. Pachetul include solver-e pentru ecuații diferențiale ordinare, parțiale, stochastice și cu întârziere. Pentru probleme nestiff, metode precum `Tsit5` și `Vern7/9` sunt folosite frecvent, iar pentru probleme stiff există familii precum `Rodas`. Ecosistemul SciML integrează și diferențiere automată, ceea ce îl face relevant pentru estimarea parametrilor și identificarea sistemelor dinamice.

3.1.3 Python

Istoric și filosofie

Python a fost creat de Guido van Rossum și lansat în 1991 [13]. Filosofia sa, rezumată în *Zen of Python* (`PEP 20`), accentuează lizibilitatea și simplitatea codului. Python a devenit limbajul dominant în știința datelor și machine learning datorită ecosistemului său extins și comunității masive.

Caracteristici tehnice și ecosistem științific

Python este un limbaj interpretat, dinamic tipizat, multi-paradigmă. Principala limitare pentru calcul intensiv este **GIL** (Global Interpreter Lock), care previne execuția paralelă a thread-urilor Python pure. Performanța este recuperată prin biblioteci care rulează cod nativ C sau Fortran în afara GIL. NumPy oferă calcul vectorizat pe array-uri dense, SciPy [14] adaugă algoritmică științifică și modulul `scipy.integrate`, iar Pandas este folosit pe scară largă pentru date tabulare. Pentru sistemul dezvoltat aici, componenta esențială este `solve_ivp`, care expune metode precum RK45, DOP853, Radau, BDF și LSODA.

Funcțiile de rezolvare a EDO în SciPy

Interfața `solve_ivp` din `scipy.integrate` este apelată printr-o funcție RHS care descrie modelul selectat. Exemplul următor folosește cazul SIR, deoarece are doar trei variabile de stare; pentru SIDARTH, aceeași interfață primește un vector de șapte compartimente și un dicționar mai mare de parametri.

```
from scipy.integrate import solve_ivp
import numpy as np

def sir_model(t, y, beta, gamma, N):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt =  beta * S * I / N - gamma * I
    dRdt =  gamma * I
    return [dSdt, dIdt, dRdt]

# Parametri
N = 1_000_000; beta = 0.3; gamma = 0.1
S0, I0, R0 = N - 1, 1, 0
t_span = (0, 160); t_eval = np.linspace(0, 160, 1601)

sol = solve_ivp(
```

```
    fun=lambda t, y: sir_model(t, y, beta, gamma, N),
    t_span=t_span,
    y0=[S0, I0, R0],
    method='RK45',
    t_eval=t_eval,
    rtol=1e-8,
    atol=1e-10
)
```

Listing 3.1: Interfața solve_ivp din SciPy pentru un sistem compartimental SIR

3.1.4 Analiză comparativă a limbajelor

Table 3.1 sintetizează principalele caracteristici ale celor trei limbaje din perspectiva utilizării pentru implementarea modelelor compartimentale SIDARTH și SIR.

Tabela 3.1: Comparație a limbajelor de programare utilizate

Criteriu	**R**	**Julia**	**Python**
Licență	Open-source	Open-source	Open-source
Paradigmă	Funcț./OOP	Multi	Multi
Compilare	Interpretat	JIT (LLVM)	Interpretat
Solver EDO	deSolve	DiffEq.jl	SciPy
Performanță rel.	Medie	Înaltă	Medie
Maturitate ecos.	Înaltă	Medie	Înaltă
Utilizare acad.	Ridicată	Medie	Ridicată

3.2 Baze de date distribuite — YugabyteDB

3.2.1 Concepte de baze de date distribuite

O *bază de date distribuită* este o colecție de date distribuite fizic pe mai multe noduri (mașini sau locații geografice), gestionată de un sistem software care oferă transparența localizării: aplicațiile interacționează cu baza de date ca și cum ar fi centralizată.

Definiție 3.1 (Teorema CAP)**.** Teorema CAP (Brewer, 2000) afirmă că un sistem distribuit nu poate garanta simultan mai mult de două dintre următoarele trei proprietăți: *consistency*, adică nodurile observă aceeași stare logică a datelor; *availability*, adică cererile valide primesc răspuns chiar în condiții de degradare; și *partition tolerance*,

adică sistemul continuă să funcționeze în prezența unor întreruperi de comunicare între noduri.

Bazele de date distribuite precum YugabyteDB, CockroachDB și Spanner optează pentru **CP** (consistency + partition tolerance), sacrificând disponibilitatea perfectă în favoarea coerenței datelor.

3.2.2 Arhitectura YugabyteDB

YugabyteDB [15] este o bază de date distribuită open-source, construită pentru a combina caracteristicile PostgreSQL cu scalabilitatea orizontală și toleranța la erori a sistemelor NoSQL. Arhitectura sa se bazează pe doi piloni principali:

1. Stratul de stocare DocDB. DocDB este motorul de stocare distribuit al YugabyteDB, bazat pe arhitectura **LSM-tree** (Log-Structured Merge-tree) derivată din **RocksDB**. Datele sunt stocate în *tablete* — unități de sharding distribuite pe noduri. Fiecare tabletă este replicată pe cel puțin trei noduri prin protocolul Raft. Scrierile trec printr-un lider, sunt replicate către nodurile următoare și sunt confirmate după ce o majoritate de replici a persistat modificarea.

2. Nivelurile de interfață API. Interfața **YSQL** este compatibilă cu PostgreSQL și suportă SQL, tranzacții ACID distribuite, join-uri și indecși. Interfața **YCQL** este compatibilă cu Apache Cassandra și este orientată spre workload-uri wide-column. În această lucrare este folosită interfața YSQL, pentru compatibilitatea cu driver-ele PostgreSQL disponibile în Go, R, Julia și Python.

3.2.3 Sharding și replicare

YugabyteDB suportă sharding prin hash și sharding pe intervale. Hash sharding distribuie uniform cheia primară pe tablete și reduce riscul de noduri fierbinți, dar este mai puțin potrivit pentru scanări ordonate pe intervale. Range sharding păstrează vecinătatea valorilor și este util pentru serii temporale, însă poate concentra scrierile dacă toate valorile noi ajung în același interval.

Observație 3.2. Pentru datele epidemiologice cu interogări frecvente pe intervale de date (serii temporale), range sharding pe coloana `timestamp` sau `date` este strategia recomandată, împreună cu un index secundar pe codul bolii și/sau regiune geografică.

3.3 Orchestrarea containerelor — Kubernetes

3.3.1 Containere și nevoia de orchestrare

Containerizarea (Docker) permite împachetarea unei aplicații împreună cu toate dependențele sale într-o unitate portabilă și reproductibilă. Pe măsură ce sistemele cresc în com-

plexitate și numărul de containere depășește câteva zeci, gestionarea manuală devine infezabilă: unde rulează fiecare container? Cum se reporneşte dacă cade? Cum se scalează?

Kubernetes (K8s) [16, 17] este platforma open-source de orchestrare a containerelor, inițial dezvoltată la Google pe baza experienței cu sistemul intern Borg, și donată CNCF (Cloud Native Computing Foundation) în 2014.

3.3.2 Arhitectura Kubernetes

Un cluster Kubernetes este împărțit între planul de control și nodurile worker. Planul de control expune API-ul clusterului prin `kube-apiserver`, păstrează starea în `etcd`, programează pod-urile prin `kube-scheduler` și rulează controllerele care aduc starea curentă spre starea dorită. Pe nodurile worker, `kubelet` gestionează ciclul de viață al pod-urilor, `kube-proxy` configurează accesul de rețea la servicii, iar container runtime-ul pornește efectiv containerele.

3.3.3 Resurse Kubernetes relevante

Resursele folosite cel mai des sunt `Pod`, `Deployment`, `StatefulSet`, `Service`, `PersistentVolumeClaim`, `ConfigMap` și `Secret`. Pentru API și frontend sunt suficiente Deployment-uri, deoarece instanțele nu păstrează stare locală. Pentru YugabyteDB este necesar un StatefulSet, care oferă identitate stabilă și volume persistente pentru fiecare replică.

3.4 Backend API — Go

3.4.1 Caracteristici și filosofia limbajului Go

Go (cunoscut și ca Golang) este un limbaj de programare open-source creat la Google de Robert Griesemer, Rob Pike și Ken Thompson, cu prima versiune stabilă în 2012 [18]. Go a fost proiectat pentru productivitate la scară mare, cu accent pe simplitate, compilare rapidă și concurență structurată.

Pentru un API backend, Go are câteva proprietăți utile: compilează rapid la cod mașină, produce binare statice ușor de containerizat, oferă concurență prin goroutine și canale, iar biblioteca standard include un pachet HTTP suficient de solid pentru servicii REST. Interfețele structurale reduc cuplarea dintre module, iar garbage collectorul concurrent menține latențe acceptabile pentru un API care nu face calcul numeric intensiv în procesul HTTP.

3.4.2 Ecosistem pentru API-uri REST

Deși `net/http` din biblioteca standard este suficient pentru API-uri simple, proiectul folosește routerul `chi`, o bibliotecă idiomatică și redusă ca suprafață, care oferă rutare ierarhică, middleware-uri compozabile și integrare directă cu tipurile standard `http.Handler`. Această alegere evită un framework mai greu și se potrivește cu dimensiunea backend-ului implementat.

3.5 Frontend — Vue.js

3.5.1 Arhitectura Vue 3 și Composition API

Vue.js [19] este un framework JavaScript progresiv pentru construirea interfețelor utilizator, creat de Evan You în 2014. Vue 3 (2020) introduce **Composition API**, o alternativă la Options API care permite organizarea logicii prin funcționalitate (nu prin tip — data, methods, computed) și reutilizarea logicii prin *composables*.

Arhitectura Vue 3 combină un sistem reactiv bazat pe `Proxy`, un Virtual DOM pentru reconcilierea arborelui de interfață și Single File Components, în care template-ul, logica TypeScript și stilurile CSS sunt grupate în fișiere `.vue`. Pentru aplicația de față, această structură permite păstrarea componentelor de formular, navigare și grafic în unități ușor de urmărit.

3.5.2 Ecosistem și biblioteci utilizate

În implementare sunt folosite Vue Router pentru rutare client-side, Pinia pentru starea globală, Axios pentru apelurile REST și Chart.js, prin `vue-chartjs`, pentru reprezentarea curbelor compartimentale ale modelului selectat. Componenta de grafic construiește dinamic seriile în funcție de cheile prezente în obiectul JSONB `compartments`, astfel încât același cod afișează atât traiectoriile $S(t)$, $I(t)$, $R(t)$ ale modelului SIR, cât și cele șapte traiectorii ale modelului SIDARTH. Aceste biblioteci acoperă nevoile aplicației fără a introduce un framework frontend suplimentar peste Vue.

Capitolul 4

Arhitectura Sistemului E-Medicine

4.1 Vedere de ansamblu

Sistemul e-medicine propus în această lucrare este o aplicație distribuită, organizată în componente independente, care permite utilizatorilor să încarce seturi de date epidemiologice reale, să declanșeze simulări SIDARTH sau SIR în trei limbaje de programare distincte — R, Julia și Python — și să compare rezultatele atât din punct de vedere numeric, cât și al performanței computaționale. Arhitectura globală este organizată în patru straturi funcționale, ilustrate în Fig. 4.1.

Stratul de prezentare constă dintr-o aplicație web single-page (SPA) implementată în Vue.js, servită printr-un container Nginx. **Stratul API** este un server REST implementat în Go, responsabil cu autentificarea prin JWT, orchestrarea job-urilor de simulare și medierea accesului la baza de date. **Stratul de calcul** cuprinde trei microservicii dedicate — numite *runnere* — fiecare implementând același contract de model compartimental într-un limbaj distinct; acestea comunică asincron cu API-ul printr-o coadă de mesaje Redis. **Stratul de stocare** este asigurat de YugabyteDB, care persistă atât datele epidemiologice de intrare, cât și rezultatele simulărilor. În mediul local, componentele sunt pornite prin Docker Compose; pentru un deployment de producție, aceeași separare a responsabilităților permite descrierea resurselor în Kubernetes.

Fluxul principal de date începe cu autentificarea utilizatorului și obținerea unui token JWT, după care acesta poate încărca un fișier CSV cu date epidemiologice; API-ul validează structura fișierului, stochează metadatele în tabelul `datasets` și înregistrările brute în `epidemiological_records`. La inițierea unei simulări, utilizatorul specifică modelul, limbajul dorit, parametrii, populația N și orizontul $t_{\max}$; API-ul creează o înregistrare în `simulation_runs` cu statusul `pending` și publică un mesaj JSON în coada Redis corespunzătoare limbajului ales. Runner-ul consumă job-ul, execută simularea cerută, scrie traiectoriile compartimentale în `simulation_results` și actualizează

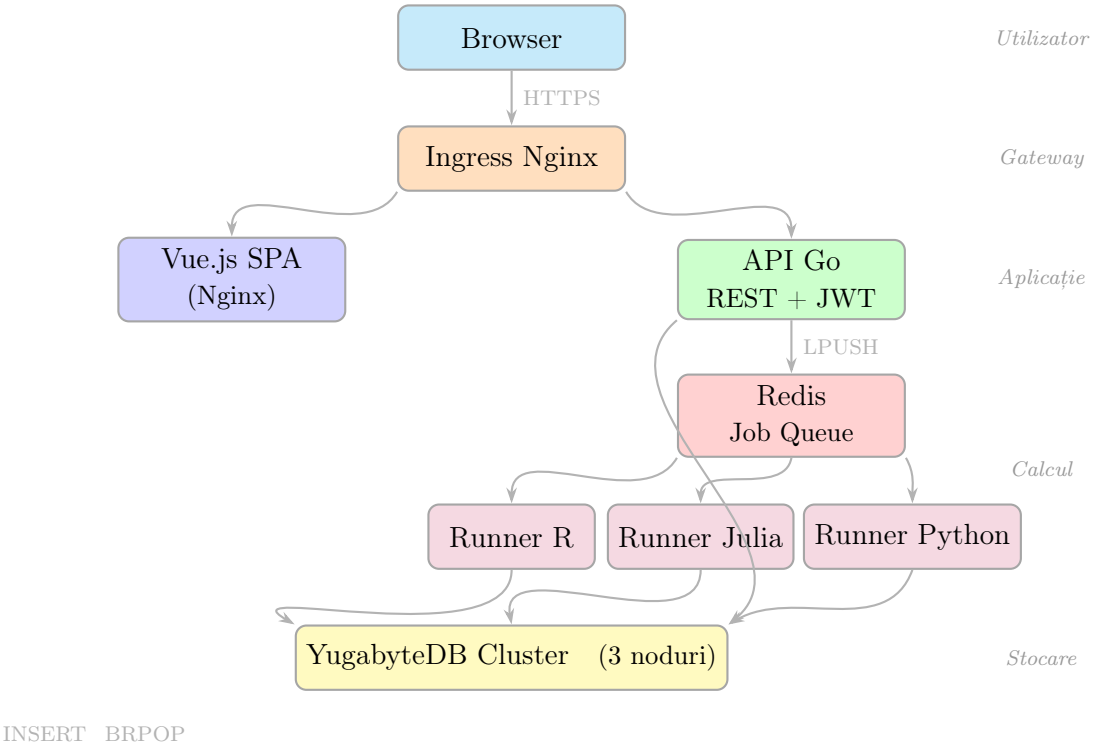


Figura 4.1: Arhitectura globală a sistemului e-medicine. Săgețile indică direcția de flux a datelor: browserul accesează aplicația prin Ingress, care rutează cererile către SPA sau API; API-ul publică job-uri în Redis și le consumă runner-ele dedicate; atât API-ul, cât și runner-ele accesează același cluster YugabyteDB.

statusul la `completed`. Frontend-ul interoghează periodic API-ul și afișează graficele imediat ce simularea este finalizată.

4.2 Cerințe funcționale și non-funcționale

4.2.1 Cerințe funcționale

Prima cerință funcțională privește **autentificarea și autorizarea**: utilizatorii trebuie să se poată înregistra și autentifica, iar toate operațiile asupra datelor și simulărilor sunt protejate prin token JWT cu durată de valabilitate configurabilă.

Gestionarea seturilor de date epidemiologice presupune că utilizatorul poate încărca fișiere CSV ce conțin serii temporale epidemiologice — date calendaristice, numere de cazuri, populație, regiune geografică. Sistemul validează structura, calculează numărul de înregistrări și stochează datele persistent în baza de date.

Configurarea și declanșarea simulărilor permite utilizatorului să selecteze un set de date, să aleagă limbajul de execuție (R, Julia sau Python) și să introducă tipul modelului, parametrii și dimensiunea populației N. Sistemul execută simularea asincron, printr-un runner dedicat, și notifică finalizarea fără a bloca interfața.

Vizualizarea rezultatelor presupune afișarea curbelor compartimentale ca gra-

fice interactive, indiferent dacă simularea are trei compartimente, ca SIR, sau șapte compartimente, ca SIDARTH.

Compararea inter-limbaj permite utilizatorului să selecteze mai multe simulări efectuate pe același set de date, cu aceiași parametri, în limbaje diferite, și să compare vizual atât forma curbelor, cât și metricile de performanță (timp de execuție, memorie utilizată).

Exportul rezultatelor oferă posibilitatea de a descărca seria temporală a unei simulări în format CSV.

Monitorizarea benchmark-urilor înseamnă că sistemul colectează automat timpii de execuție (wall-clock și CPU) și consumul de memorie pentru fiecare rulare și le expune printr-un endpoint dedicat, accesibil în interfața de comparare.

4.2.2 Cerințe non-funcționale

Din punct de vedere al performanței, sistemul trebuie să păstreze scurte operațiile sincrone ale API-ului. Crearea unei simulări nu execută modelul în procesul HTTP, ci doar validează cererea, persistă metadatele și publică job-ul în Redis. Durata simulării este tratată separat, la nivel de runner.

Disponibilitatea depinde de modul de deployment. În varianta Docker Compose, sistemul este potrivit pentru dezvoltare și testare locală. Într-un deployment Kubernetes, API-ul poate fi replicat, iar YugabyteDB poate rula cu factor de replicare 3, tolerând pierderea unui nod fără pierdere de date confirmate.

Scalabilitatea este obținută prin design-ul stateless al API-ului și prin arhitectura bazată pe coadă: numărul de replici ale oricărui runner poate fi crescut orizontal prin simpla modificare a câmpului `replicas` în Deployment-ul Kubernetes, fără modificări de cod; YugabyteDB suportă adăugarea de noduri fără downtime.

Securitatea implică protejarea tuturor endpointurilor sensibile prin JWT, gestionarea secretelor (chei JWT, parole DB) prin obiecte Kubernetes `Secret` — montate ca variabile de mediu, nu incluse în imaginile Docker — și aplicarea principiului privilegiilor minime la nivelul ServiceAccount-urilor Kubernetes.

Extensibilitatea garantează că adăugarea unui nou limbaj sau a unui nou model compatibil necesită doar crearea unui nou container runner care respectă interfața de comunicare cu coada Redis, fără modificarea API-ului sau a schemei bazei de date.

Portabilitatea este asigurată prin containerizarea (Docker) a tuturor componentelor și descrierea lor declarativă prin manifeste Kubernetes, permițând deployment pe orice furnizor de cloud sau infrastructură on-premises care suportă Kubernetes 1.28+.

4.3 Arhitectura bazei de date distribuite

4.3.1 Schema relațională

Baza de date a sistemului este structurată în șase tabele principale, ale căror relații sunt ilustrate în Fig. 4.2.

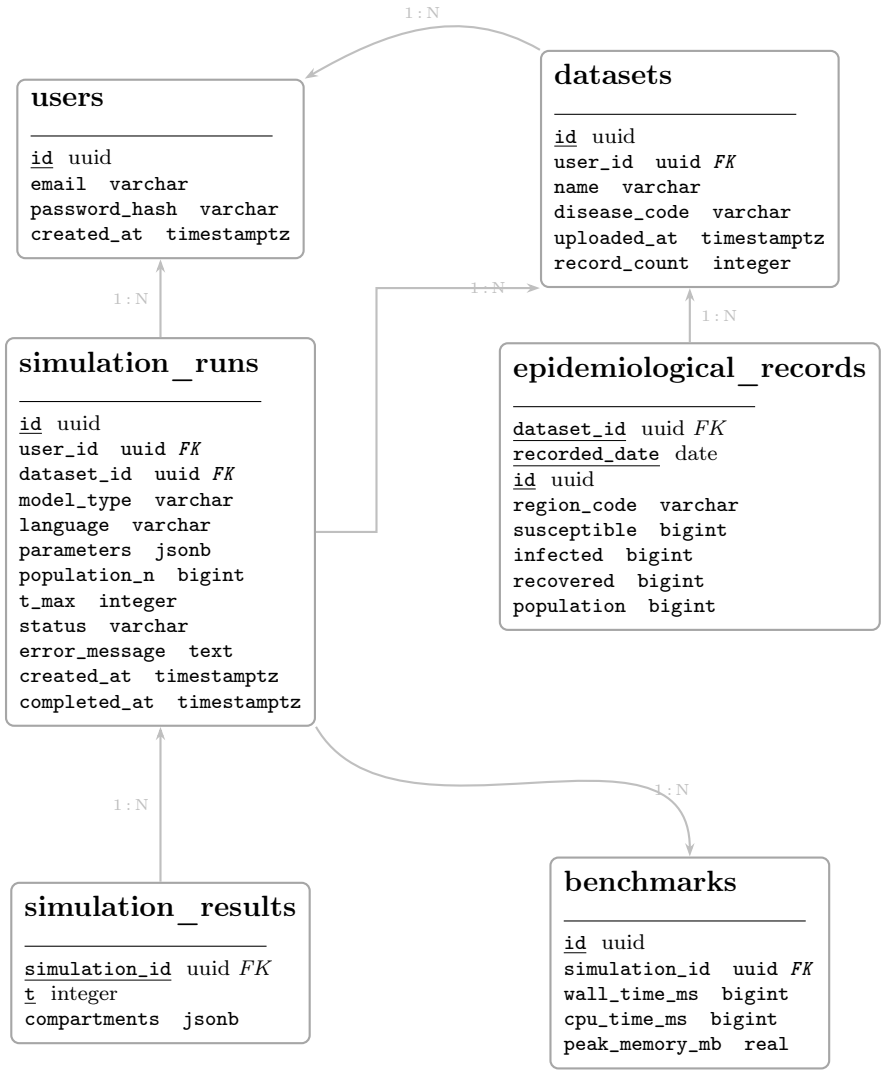


Figura 4.2: Diagrama entitate-relație a bazei de date. PK-urile sunt marcate cu aldine și subliniere; FK-urile sunt marcate cu *FK*; raportul de cardinalitate 1 : N este indicat pe fiecare relație.

Tabelul `users` stochează datele de autentificare ale utilizatorilor; parola nu este reținută în clar, ci ca hash bcrypt cu factor de cost 12. Tabelul `datasets` conține metadatele seturilor de date epidemiologice încărcate: autorul, codul bolii (ex. `COVID-19`, `SARS`), data încărcării și numărul total de înregistrări. Tabelul `epidemiological_records` stochează valorile brute ale seriei temporale; câmpurile `susceptible`, `infected` și `recovered` conțin valorile observate sau valorile inițiale folosite la calibrarea parametrilor, iar `region_code` urmează standardul ISO 3166-2. Tabelul `simulation_runs`

reprezintă job-ul de simulare și include tipul modelului (`sidarth` sau `sir`), parametrii în format JSONB, populația, orizontul temporal, limbajul ales, statusul execuției (`pending`, `running`, `completed`, `failed`), câmpul `error_message` pentru capturarea erorilor de execuție și timestampurile de creare și finalizare. Tabelul `simulation_results` stochează seria temporală de ieșire: pentru fiecare pas de timp $t \in \{0, 1, \ldots, t_{\max}\}$, câmpul `compartments` conține un obiect JSONB cu valorile compartimentelor modelului. Cheia primară compusă `(simulation_id, t)` identifică unic fiecare punct și asigură interogări eficiente ale seriei complete. Tabelul `benchmarks` reține metricile de performanță colectate automat la finalul fiecărei rulări: timpul wall-clock, timpul CPU și consumul maxim de memorie.

4.3.2 Modelul datelor epidemiologice

Datele epidemiologice sunt ingerate în sistem prin fișiere CSV cu structura minimă prezentată în Table 4.1.

Tabela 4.1: Structura minimă a fișierului CSV de date epidemiologice.

Câmp	**Tip**	**Descriere**
`date`	`DATE`	Data înregistrării (format ISO 8601, obligatoriu)
`region`	`VARCHAR`	Cod regiune ISO 3166-2 (obligatoriu)
`infected`	`BIGINT`	Număr de cazuri active (obligatoriu)
`population`	`BIGINT`	Populația totală (obligatoriu)
`recovered`	`BIGINT`	Cazuri recuperate (opțional)
`susceptible`	`BIGINT`	Populația susceptibilă estimată (opțional)

Câmpurile `susceptible` și `recovered` sunt opționale; dacă lipsesc, API-ul le estimează prin $S(0) = N - I(0)$ și $R(0) = 0$. Granularitatea temporală recomandată este zilnică; datele săptămânale sunt acceptate, dar reduc precizia estimării parametrilor. La încărcarea unui set de date, API-ul verifică că toate câmpurile obligatorii sunt prezente și au tipul corect, că relația $I(t) + R(t) \leq N$ este respectată pentru fiecare înregistrare, că seria temporală este ordonată cronologic și nu conține goluri mai mari de 14 zile consecutive (condiție tratată ca avertisment, nu eroare critică), și că fiecare cod de regiune este nevid. Validarea formală față de un registru ISO nu este inclusă în implementarea curentă.

4.3.3 Strategia de partiționare și replicare în YugabyteDB

YugabyteDB utilizează un model de stocare bazat pe *tablets* (echivalentul shard-urilor), gestionate prin protocolul de consens Raft [20]. Clusterul este configurat cu factor de

replicare $RF = 3$, garantând că pierderea oricărui nod unic nu afectează disponibilitatea sau consistența datelor.

Tabelul `epidemiological_records` este proiectat pentru *range sharding* pe cheia compusă `(dataset_id, recorded_date)`. Această strategie optimizează interogările de tip *range scan* pe interval temporal, care reprezintă cel mai frecvent pattern de acces — de exemplu, extragerea seriei temporale complete a unui set de date. În mediul de producție, partiționarea inițială se declară explicit prin clauza `SPLIT AT VALUES`, prezentată mai jos; migrația de bază aplică schema fără presplitting, lăsând YugabyteDB să partitioneze automat pe măsura creșterii datelor.

```
CREATE TABLE epidemiological_records (
    id              UUID DEFAULT gen_random_uuid(),
    dataset_id      UUID NOT NULL REFERENCES datasets(id),
    recorded_date DATE NOT NULL,
    region_code     VARCHAR(10) NOT NULL,
    susceptible     BIGINT,
    infected        BIGINT NOT NULL,
    recovered       BIGINT,
    population      BIGINT NOT NULL,
    PRIMARY KEY (dataset_id, recorded_date, id)
) SPLIT AT VALUES (
    ('00000000-0000-0000-0000-000000000000', '2020-01-01'),
    ('00000000-0000-0000-0000-000000000000', '2021-01-01'),
    ('00000000-0000-0000-0000-000000000000', '2022-01-01')
);
```

Listing 4.1: DDL de producție al tabelului `epidemiological_records` cu range sharding presplit pe cheie temporală.

Tabelul `simulation_results` utilizează, în schimb, hash sharding pe `simulation_id`, distribuind uniform rândurile între tablets, deoarece accesul la rezultate este întotdeauna filtrat după un `simulation_id` specific și nu necesită range scans inter-simulări.

Fiecare tablet are un lider Raft ales dintre cele trei replici; scrierile trec prin lider (*strong consistency*) și sunt replicate sincron pe celelalte două noduri înainte de confirmare. Parametrul `ysql_num_shards_per_tserver` este setat la 4 (față de valoarea implicită 8) pentru a reduce supraîncărcarea managementului de tablets pe un cluster de dimensiuni mici.

4.4 Arhitectura API-ului backend Go

4.4.1 Structura modulară a proiectului

Proiectul backend este organizat conform convențiilor comunității Go (*Standard Go Project Layout*) [21]:

```
emedicine-api/
+-- cmd/
|   `-- server/
|       `-- main.go        # Entry point: initializare, injectie dependinte
+-- internal/
|   +-- api/
|   |   +-- handlers/      # Handler HTTP per resursa (datasets, sims, auth)
|   |   +-- middleware/    # JWT validation, logging, CORS, rate-limit
|   |   `-- router/        # Configurare rute (chi router)
|   +-- db/
|   |   +-- migrations/    # Fisiere SQL versionate (golang-migrate)
|   |   `-- repository/    # Interfete si implementari repository pattern
|   +-- runner/
|   |   +-- dispatcher.go # Publicare job-uri in Redis
|   |   `-- queue.go       # Definitia mesajului JSON al job-ului
|   `-- models/            # Struct-uri de domeniu (User, Dataset, Run etc.)
```

Listing 4.2: Structura de directoare a proiectului backend Go.

Arhitectura internă respectă pattern-ul în trei straturi. **Handler layer** parsează cererea HTTP, validează input-ul și serializează răspunsul JSON, fără a conține logică de business. **Service layer** orchestrează tranzacțiile, aplică regulile domeniului și publică job-uri în Redis. **Repository layer** abstractizează accesul la YugabyteDB prin interfețe Go, permițând testarea izolată a serviciilor fără a depinde de baza de date reală. Dependențele sunt injectate explicit prin constructori, conform idiomelor limbajului Go, fără un container IoC.

4.4.2 Mecanismul de autentificare JWT

Autentificarea se bazează pe JSON Web Tokens (RFC 7519) [22], utilizând algoritmul de semnătură `HS256` (HMAC-SHA256). Cheia secretă este generată la instalare și stocată ca Kubernetes Secret, montată în containerul API ca variabilă de mediu — nu este inclusă în imaginea Docker. La autentificare, clientul trimite credențialele la `POST /api/auth/login`; API-ul verifică hash-ul bcrypt al parolei din baza de date și, în caz de succes, returnează un JWT cu payload `{sub: user_id, exp: now+24h, iat: now}`. Clientul include token-ul în fiecare cerere ulterioară prin header-ul `Authorization: Bearer <token>`, iar middleware-ul JWT îl validează — verificând semnătura și expirerea — fără a accesa baza de date. Absența oricărei stări pe server este esențială pentru scalabilitatea orizontală: oricare replică a API-ului poate valida orice token, fără sincronizare între instanțe.

4.4.3 Diagrama endpointurilor REST

Table 4.2 prezintă endpointurile REST expuse de API.

Tabela 4.2: Endpointurile REST ale API-ului backend.

Metodă	**Cale**	**Descriere**	**Auth**
POST	`/api/auth/register`	Înregistrare utilizator nou	Nu
POST	`/api/auth/login`	Autentificare, returnează JWT	Nu
GET	`/api/datasets`	Lista seturilor de date ale utilizatorului	DA
POST	`/api/datasets`	Încărcare fișier CSV nou	DA
GET	`/api/datasets/{id}`	Detalii și statistici set de date	DA
DELETE	`/api/datasets/{id}`	Ștergere set de date	DA
POST	`/api/simulations`	Inițiere simulare nouă	DA
GET	`/api/simulations`	Lista simulărilor utilizatorului	DA
GET	`/api/simulations/{id}`	Status și metadate simulare	DA
GET	`/api/simulations/{id}/results`	Seria temporală compartimentală	DA
GET	`/api/simulations/{id}/export`	Export CSV rezultate	DA
GET	`/api/benchmarks`	Benchmark-uri agregate per model și limbaj	DA
GET	`/api/health`	Health check (liveness probe K8s)	Nu

Răspunsurile API urmează o convenție simplificată inspirată din *JSON:API*: obiectul de nivel superior conține fie câmpul `data` (la succes), fie `error` cu un cod numeric și un mesaj uman-citibil (la eșec), permițând tratarea uniformă a erorilor în clientul frontend.

4.4.4 Integrarea cu runner-ele de algoritmi

Comunicarea dintre API-ul Go și runner-e este asincronă, mediată de Redis printr-un model *producer–consumer*. Când utilizatorul inițiază o simulare, API-ul publică un obiect JSON în lista Redis `epimodel:jobs:<language>` (ex. `epimodel:jobs:r`):

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id":    "7b4e1a02-...",
  "model_type":    "sidarth",
  "parameters":    {"alpha": 0.57, "beta": 0.011, "gamma":
   0.456},
  "population_n":  1000000,
  "t_max":         365,
  "initial":       {"s": 998999.0, "i": 1001.0, "d": 0.0,
                    "a": 0.0, "r": 0.0, "t": 0.0, "h": 0.0},
  "db_dsn":        "postgres://emedicine:***@yb-tserver-
   service.yugabyte.svc.cluster.local:5433/emedicine"
}
```

Listing 4.3: Exemplu mesaj JSON publicat în coada Redis la inițierea unei simulări.

Fiecare runner execută o buclă `BRPOP` blocantă pe coada sa dedicată. La primirea unui job, runner-ul actualizează statusul simulării la `running`, execută modelul cerut prin `model_type` (detaliat în Chapter 5), inserează rezultatele în `simulation_results` prin inserări batch de $t_{\max} + 1$ rânduri, înregistrează metricile de performanță în `benchmarks`, iar în final actualizează statusul la `completed` — sau la `failed`, cu mesajul de eroare stocat, dacă a apărut o excepție. Existența cozilor separate per limbaj (`epimodel:jobs:r`, `epimodel:jobs:julia`, `epimodel:jobs:python`) permite scalarea independentă a fiecărui runner în funcție de cerere, fără a modifica logica de rutare a API-ului.

4.5 Arhitectura frontend Vue.js

4.5.1 Structura componentelor

Frontend-ul este implementat ca o aplicație Vue.js 3 cu Composition API, folosind Vite ca bundler și TypeScript pentru siguranța tipurilor:

```
emedicine-ui/
‘-- src/
    +-- pages/
    |   +-- LoginPage.vue
    |   +-- RegisterPage.vue
    |   +-- DashboardPage.vue       # Sumar: nr. datasets, simulari recente
    |   +-- DatasetsPage.vue        # Lista si upload datasets
    |   +-- DatasetDetailPage.vue
    |   +-- NewSimulationPage.vue
    |   +-- SimulationResultPage.vue
    |   +-- ComparePage.vue         # Comparare simulari inter-limbaj
    |   ‘-- BenchmarksPage.vue
    +-- components/
    |   +-- CompartmentChart.vue    # Grafic Chart.js pentru compartimente
    |   +-- BenchmarkTable.vue
    |   +-- ParameterForm.vue       # Formular model, parametri, t_max
    |   +-- DatasetUpload.vue       # Drag-and-drop CSV
    |   +-- SimulationStatus.vue    # Badge status cu polling
    |   ‘-- NavBar.vue
    +-- stores/
    |   +-- auth.store.ts
    |   +-- datasets.store.ts
    |   +-- simulations.store.ts
    |   ‘-- benchmarks.store.ts
    +-- api/
    |   ‘-- client.ts               # Axios instance cu JWT interceptor
    ‘-- router/
        ‘-- index.ts                # Vue Router cu navigation guards
```

Listing 4.4: Structura de directoare a proiectului Vue.js.

4.5.2 Fluxul de date și state management

Gestionarea stării globale este realizată cu Pinia [23], biblioteca oficială de state management pentru Vue.js 3. Fiecare store encapsulează starea, acțiunile (apeluri API

asincrone) și getters (date derivate) pentru un domeniu specific.

La autentificare, `authStore` primește JWT-ul de la API și îl stochează în `localStorage` pentru persistență între sesiuni. Un interceptor Axios adaugă automat header-ul `Authorization: Bearer` la fiecare cerere ulterioară, iar navigation guards Vue Router redirecționează utilizatorii neautentificați spre pagina de login.

Fluxul de simulare începe din pagina `NewSimulationPage`, care apelează `simulationsStore.creat` cu parametrii formularului; store-ul primește înapoi `simulation_id` și componenta `SimulationStatus` pornește un polling la interval de 2 secunde pe `GET /api/simulations/{id}`, oprindu-se când statusul devine `completed` sau `failed`. La finalizare, `SimulationResultPage` încarcă seria temporală și o pasează componentei `CompartmentChart`, care randează graficul interactiv pe baza cheilor prezente în obiectul `compartments`.

Pagina `ComparePage` permite selectarea a 2–3 simulări finalizate. Pentru fiecare rulare selectată, store-ul încarcă metadatele și seria temporală prin cereri paralele, iar `CompartmentChart` le suprapune în același grafic, diferențiindu-le prin stil de linie (continuu, întrerupt, punctat).

4.6 Infrastructura Kubernetes

4.6.1 Topologia cluster-ului

Clusterul Kubernetes este compus din trei noduri: un nod de control plane și doi workeri, conform Fig. 4.3. Configurația hardware minimă recomandată este de 4 vCPU, 8 GiB RAM și 50 GiB SSD per nod, cu 8 vCPU și 16 GiB RAM recomandate pentru nodurile worker care găzduiesc runner-ele și instanțele YugabyteDB.

Toate resursele aplicației sunt grupate în namespace-ul `emedicine`, separat de namespace-ul `yugabyte` al bazei de date, facilitând aplicarea politicilor de rețea (NetworkPolicy) și controlul accesului bazat pe roluri (RBAC) la nivel de namespace.

4.6.2 Deployment YugabyteDB pe Kubernetes

YugabyteDB este deploiat prin Helm chart-ul oficial [24], configurând câte un StatefulSet cu 3 replici pentru componentele `yb-master` și `yb-tserver`:

```
helm repo add yugabytedb https://charts.yugabyte.com
helm install yugabyte yugabytedb/yugabyte \
  --namespace yugabyte --create-namespace \
  --set replicas.master=3 \
  --set replicas.tserver=3 \
  --set storage.master.size=10Gi \
  --set storage.tserver.size=50Gi \
  --set resource.tserver.requests.cpu=2 \
```

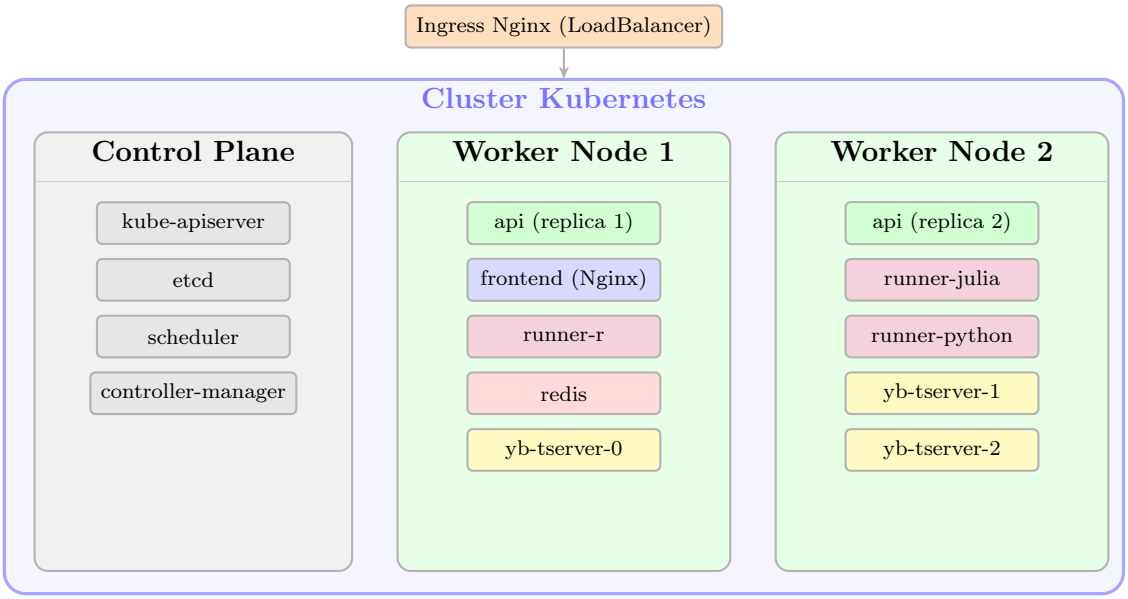


Figura 4.3: Topologia cluster-ului Kubernetes. Ingressul este expus ca LoadBalancer Service în fața clusterului. Worker Node 1 găzduiește frontend-ul, primul runner și prima instanță YugabyteDB tserver; Worker Node 2 găzduiește celelalte două runner-e și doi tserveri.

```
  --set resource.tserver.requests.memory=4Gi \
  --set gflags.tserver.ysql_enable_auth=true
```

Listing 4.5: Instalare YugabyteDB cu Helm.

Helm chart-ul creează automat StatefulSet-urile `yb-master` și `yb-tserver` cu câte 3 pod-uri fiecare, un `PersistentVolumeClaim` per pod (pe StorageClass `fast-ssd` dacă este disponibil, altfel `standard`), un headless Service pentru DNS intern stabil al pod-urilor (ex. `yb-tserver-0.yb-tservers.yugabyte.svc.cluster.local`), și un Service de tip ClusterIP pe portul 5433 (YSQL) utilizat de API-ul Go pentru conexiunea la baza de date.

4.6.3 Deployment API Go și frontend Vue.js

Imaginea Docker a API-ului este construită cu un *multi-stage build*: primul stage compilează binarul Go static (`CGO_ENABLED=0`), iar al doilea stage copiază doar binarul într-o imagine `distroless/static`, obținând o imagine finală de aproximativ 15 MiB. Deployment-ul Kubernetes configurează 2 replici, un liveness probe pe `GET /api/health` și un readiness probe care verifică conectivitatea la YugabyteDB înainte de a admite trafic.

Aplicația Vue.js este compilată în fișiere statice (`npm run build`) și servită printr-un container Nginx configurat ca SPA: toate rutele necunoscute sunt redirecționate la `index.html`, permițând Vue Router să gestioneze navigarea côté client.

Un singur Ingress resource (nginx-ingress controller) rutează traficul extern: cererile cu prefix `/api` sunt direcționate spre Service-ul `api-service` pe portul 8080, iar toate celelalte spre Service-ul `frontend-service` pe portul 80. În mediul de producție, terminarea TLS se configurează la nivel de Ingress prin cert-manager cu certificate Let's Encrypt; în mediul local (k3d), Ingress-ul este expus fără TLS.

Capitolul 5

Implementarea modelelor SIDARTH și SIR

5.1 Datele de intrare și inițializarea compartimentelor

Datele epidemiologice utilizate de sistem provin din setul public *Our World in Data* (OWID) COVID-19 [25]. Fișierele brute nu conțin direct toate compartimentele necesare unui model SIDARTH, ci serii temporale agregate, cum ar fi cazurile noi, cazurile cumulate, decesele cumulate și populația. Din acest motiv, etapa de preprocesare nu pretinde că recuperează starea epidemiologică completă a populației. Ea construiește o stare inițială compatibilă cu sistemul software, folosind datele observabile drept punct de pornire pentru explorarea numerică a scenariilor.

Scriptul `scripts/preprocess_owid.py` reindexează fiecare țară pe un calendar zilnic continuu, completează cumulativii prin forward-fill și înlocuiește valorile negative din `new_cases` cu zero, deoarece acestea apar de obicei ca ajustări retrospective ale raportărilor. Numărul de infectați activi este aproximat printr-o fereastră mobilă de 14 zile:

$$I(t) = \sum_{\tau=t-\Delta}^{t} \text{new_cases}(\tau), \qquad \Delta = 14. \tag{5.1}$$

Compartimentul *removed* este aproximat prin

$$R_{\text{removed}}(t) = \text{total_cases}(t) - I(t), \tag{5.2}$$

iar susceptibilii se obțin din bilanțul populației,

$$S(t) = N - I(t) - R_{\text{removed}}(t). \tag{5.3}$$

Pentru modelul SIR, aceste trei valori pot fi folosite direct ca (S_0, I_0, R_0). Pentru SIDARTH, aceeași informație este distribuită într-o stare inițială mai bogată: cazurile

active observabile sunt plasate în compartimentul I, persoanele ieșite din infecție sunt plasate în H, iar compartimentele D, A, R și T pornesc de la zero. Această alegere este deliberat conservatoare. Ea nu afirmă că distribuția clinică reală este cunoscută, ci oferă o condiție inițială reproductibilă, potrivită pentru compararea implementărilor și pentru analiza sensibilității la parametri.

Tabela 5.1: Maparea datelor observabile în condiții inițiale pentru modelele suportate.

Model	**Compartimente inițiale**	**Interpretare**
SIR	S_0, I_0, R_0	model agregat cu stare minimă
SIDARTH	S_0, I_0, $D_0 = A_0 = R_0 = T_0 = 0$, H_0	model clinic stratificat

5.2 Contractul comun de simulare

Pivotarea către SIDARTH a impus schimbarea contractului intern dintre API și runnere. Versiunea inițială a sistemului transmitea doar parametrii β și γ și presupunea implicit că rezultatul conține curbele S, I și R. Noua versiune introduce câmpul `model_type`, un obiect `parameters` și un obiect `initial`. În acest fel, același flux poate executa SIDARTH ca model principal și SIR ca model secundar, fără a modifica interfața de cozi sau mecanismul de benchmark.

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id": "7b4e1a02-...",
  "model_type": "sidarth",
  "language": "python",
  "parameters": {
    "alpha": 0.57, "beta": 0.011, "gamma": 0.456, "delta":
   0.011,
    "epsilon": 0.171, "zeta": 0.125, "lambda": 0.034,
    "eta": 0.125, "rho": 0.034, "theta": 0.370,
    "mu": 0.017, "kappa": 0.017, "nu": 0.027,
    "xi": 0.017, "sigma": 0.017
  },
  "initial": {
    "s": 19000000, "i": 1200, "d": 0, "a": 0,
    "r": 0, "t": 0, "h": 18000
  },
  "population_n": 19019200,
  "t_max": 160,
  "db_dsn": "postgres://..."
```

```
}
```

Listing 5.1: Structura job-ului publicat în Redis pentru o simulare SIDARTH.

Coada Redis a fost redenumită din `sir:jobs:<language>` în `epimodel:jobs:<language>`. Schimbarea este mică în cod, dar importantă conceptual: runner-ul nu mai este o implementare exclusivă SIR, ci un executor pentru modele compartimentale compatibile cu schema comună. Rezultatele sunt stocate în `simulation_results` ca obiect JSONB `compartments`, nu ca trei coloane fixe. Pentru o rulare SIDARTH, un punct temporal conține cheile `s`, `i`, `d`, `a`, `r`, `t` și `h`; pentru SIR, același câmp conține doar `s`, `i` și `r`.

5.3 Implementarea modelului SIDARTH

Modelul implementat în această lucrare este o variantă SIDARTH derivată din familia SIDARTHE propusă de Giordano et al. [5]. Se păstrează compartimentele S, I, D, A, R, T și H, iar compartimentul E al deceselor este omis pentru a concentra aplicația pe dinamica infecției, diagnosticării, severității și vindecării. Ecuația pentru T folosește astfel doar ieșirea către H, prin rata σ, nu și termenul de mortalitate τ.

În Python, nucleul modelului este o funcție RHS compatibilă cu `scipy.integrate.solve_ivp`. Ea primește vectorul de stare, dicționarul de parametri și populația totală:

```
def sidarth_rhs(t, y, params, n):
    s, i, d, a, r, threatened, h = y
    force = (
        params["alpha"] * i +
        params["beta"] * d +
        params["gamma"] * a +
        params["delta"] * r
    ) / n
    new_infections = s * force

    return [
        -new_infections,
        new_infections - (params["epsilon"] + params["zeta"] +
    params["lambda"]) * i,
        params["epsilon"] * i - (params["eta"] + params["rho"
   ]) * d,
        params["zeta"] * i - (params["theta"] + params["mu"] +
    params["kappa"]) * a,
        params["eta"] * d + params["theta"] * a - (params["nu"
   ] + params["xi"]) * r,
```

```
        params["mu"] * a + params["nu"] * r - params["sigma"]
    * threatened,
        params["lambda"] * i + params["rho"] * d + params["
    kappa"] * a
            + params["xi"] * r + params["sigma"] * threatened,
     ]
```

Listing 5.2: Funcția RHS SIDARTH în runner-ul Python.

Aceeași structură este implementată în R și Julia. În R, funcția este transmisă către `deSolve::ode()`, cu metoda `rk4`, păstrând pasul zilnic fix. În Julia, funcția respectă convenția *in-place* din `DifferentialEquations.jl`, iar integrarea folosește `Tsit5()` cu `saveat = 0:1.0:t_max`. Diferența dintre cele trei implementări nu este matematică, ci ține de ecosistemul numeric: R oferă o interfață stabilă și compactă pentru ODE-uri clasice, Julia urmărește performanța și controlul fin al solverului, iar Python oferă integrare directă cu infrastructura software și cu bibliotecile științifice uzuale.

5.4 SIR ca model secundar

Modelul SIR rămâne în sistem pentru comparații și pentru verificări de regresie. El are un rol diferit față de versiunea inițială a lucrării. Nu mai este modelul principal prin care se discută dinamica clinică a epidemiei, ci un reper minimal: dacă două limbaje produc rezultate apropiate pentru SIR, diferențele observate ulterior în SIDARTH pot fi analizate în raport cu complexitatea modelului, nu cu erori elementare de integrare sau de persistare.

În contractul comun, SIR folosește același câmp `parameters`, dar cu doar două rate, `beta` și `gamma`. Condițiile inițiale sunt transmise prin `initial = {s, i, r}`, iar rezultatele sunt stocate tot în câmpul `compartments`. Această simetrie simplifică interfața web: graficul citește cheile disponibile în fiecare punct temporal și construiește curbele fără să presupună apriori numărul de compartimente.

5.5 Persistența rezultatelor și exportul

Tabela `simulation_runs` reține metadatele execuției: utilizatorul, dataset-ul, modelul, limbajul, populația, orizontul temporal și parametrii sub formă JSONB. Tabela `simulation_results` reține câte un rând pentru fiecare zi simulată, identificat prin perechea `(simulation_id, t)`. În locul coloanelor fixe `s`, `i`, `r`, rândul conține obiectul JSONB `compartments`. Această decizie evită migrații repetate ale schemei atunci când se adaugă un model cu altă structură de stare.

```
CREATE TABLE simulation_results (
```

```
    simulation_id UUID NOT NULL REFERENCES simulation_runs(id)
    ON DELETE CASCADE,
    t             INTEGER NOT NULL,
    compartments  JSONB NOT NULL,
    PRIMARY KEY (simulation_id, t),
    CHECK (t >= 0),
    CHECK (jsonb_typeof(compartments) = 'object')
);
```

Listing 5.3: Schema generică pentru rezultatele simulărilor.

Exportul CSV este generat dinamic. API-ul inspectează cheile prezente în rezultate, le ordonează în secvența epidemiologică S, I, D, A, R, T, H și scrie doar coloanele existente pentru modelul respectiv. Prin urmare, o simulare SIR se exportă cu coloanele `t,s,i,r`, iar o simulare SIDARTH cu `t,s,i,d,a,r,t,h`. Această convenție păstrează fișierele lizibile și evită completarea artificială a unor compartimente care nu există în model.

5.6 Metrici de performanță

Toate runnerele colectează aceleași trei metrici: timpul wall-clock, timpul CPU și vârful de memorie rezidentă. Python folosește `time.perf_counter()`, `time.process_time()` și `resource.getrusage`; R folosește `proc.time()` și `/proc/self/status`; Julia folosește `time_ns()` și `Sys.maxrss()`. Valorile sunt scrise în tabela `benchmarks` după inserarea traiectoriilor, în aceeași tranzacție logică în care rularea este marcată ca `completed`. Dacă integrarea numerică sau persistarea eșuează, runner-ul marchează rularea ca `failed` și păstrează mesajul de eroare, astfel încât interfața web să poată diferenția între o simulare aflată încă în lucru și una care a eșuat definitiv.

Capitolul 6

Evaluarea Performanței și Scalabilității

6.1 Principiul evaluării

Evaluarea performanței urmărește separarea costului numeric de costurile introduse de arhitectura aplicației. În sistemul implementat, modelul nu este apelat direct din interfață, ci este executat de un runner care primește job-ul din Redis și scrie rezultatele în YugabyteDB. Prin urmare, măsurarea trebuie să țină cont de solver, de serializarea rezultatelor, de inserarea în baza de date și de tranzițiile de status ale simulării.

Unitatea experimentală este o rulare completă a unui model ales prin `model_type`. Pentru SIDARTH, vectorul de stare are șapte componente, iar pentru SIR are trei. Ambele folosesc aceeași infrastructură: API-ul creează rândul în `simulation_runs`, publică job-ul în Redis, runner-ul execută integrarea numerică, scrie seria temporală în `simulation_results` și salvează metricile în `benchmarks`. Această definiție include costul operațional real al runner-ului, nu doar apelul către solver.

6.2 Metrici colectate

Tabela 6.1: Metricile colectate pentru fiecare rulare.

Metrică	**Interpretare**
`wall_time_ms`	Durata observată de procesul runner între începutul rezolvării și finalizarea persistării rezultatelor.
`cpu_time_ms`	Timpul de procesor consumat de runner; în Julia este aproximat prin timpul elapsed, din cauza diferențelor de expunere a timpului CPU.
`peak_memory_mb`	Vârful de memorie rezidentă raportat de proces pe durata rulării.

Python măsoară timpul cu `time.perf_counter()` și `time.process_time()`, R folosește `proc.time()`, iar Julia folosește `time_ns()`. Aceste mecanisme nu sunt

identice, dar sunt disponibile nativ în fiecare runtime și permit colectarea automată fără un profiler extern.

6.3 Protocol experimental

Un protocol reproductibil trebuie să controleze încălzirea runtime-ului, starea bazei de date și concurența dintre job-uri. În Julia, prima rulare poate include costuri de compilare JIT, motiv pentru care rezultatele relevante trebuie separate între rulări de încălzire și rulări măsurate. În R și Python, costul de import al bibliotecilor este plătit la pornirea containerului, nu la fiecare job.

Pentru o campanie finală se recomandă rulări repetate pentru fiecare combinație model–limbaj, cu aceeași imagine Docker, aceleași toleranțe numerice și un singur job activ pe runner. Valorile agregate ar trebui raportate prin mediană și interval intercuartilic, deoarece timpii în containere pot avea cozi lungi produse de scheduling, I/O sau garbage collection.

6.4 Scalabilitate și complexitatea modelului

În modelele compartimentale agregate, mărimea populației N nu crește direct dimensiunea sistemului diferențial. O simulare SIDARTH pentru $N = 10^3$ și una pentru $N = 10^7$ au același număr de ecuații; se schimbă doar scara valorilor. Costul crește în schimb cu numărul de compartimente, cu orizontul temporal $t_{\max}$ și cu numărul de puncte salvate.

SIDARTH este mai costisitor decât SIR nu pentru că populația este mai mare, ci pentru că fiecare evaluare RHS calculează mai multe fluxuri și fiecare punct salvat conține mai multe valori. În plus, persistarea în JSONB introduce un cost de serializare care nu exista în schema fixă SIR, dar acest cost cumpără flexibilitate: aceeași tabelă poate stoca modele cu structuri diferite de stare.

6.5 Compararea implementărilor R, Julia și Python

Cele trei implementări nu folosesc exact același algoritm numeric. Runner-ul R folosește RK4 cu pas fix $h = 1$, Python folosește `solve_ivp` cu RK45, iar Julia folosește `Tsit5`. Comparația trebuie interpretată ca o comparație între implementări practice echivalente funcțional, nu ca un experiment izolat pe același solver. Pentru o comparație strictă ar fi necesară implementarea manuală a aceleiași metode în toate cele trei limbaje.

Avantajul abordării curente este că reflectă utilizarea naturală a fiecărui ecosistem: R prin `deSolve`, Python prin SciPy, Julia prin SciML. Rezultatele benchmarkului trebuie deci citite ca o comparație între ecosisteme de calcul științific integrate într-un sistem software, nu doar între limbaje de programare.

6.6 Acuratețe numerică

Corectitudinea rezultatelor este verificată prin conservarea populației. Pentru SIDARTH, suma $S + I + D + A + R + T + H$ trebuie să rămână egală cu N la fiecare pas salvat; pentru SIR, condiția devine $S + I + R = N$. Un al doilea criteriu este apropierea traiectoriilor obținute în cele trei limbaje pentru aceeași configurație.

O soluție de referință poate fi obținută printr-o metodă adaptivă cu toleranțe mai stricte, de exemplu `DOP853` în SciPy sau `Vern9` în Julia. În absența unei campanii numerice complete, capitolul nu susține o ierarhie finală de acuratețe între limbaje, ci definește criteriul prin care ea poate fi stabilită.

6.7 Interpretarea rezultatelor

În forma actuală, infrastructura de măsurare este prezentă, iar fiecare rulare produce metrici persistente. Pentru concluzii cantitative finale sunt necesare campanii controlate și tabele agregate pe combinații model–limbaj. Fără aceste date, afirmații precum „limbajul X este cel mai rapid” ar fi premature.

Rezultatul important în această etapă este metodologic: sistemul permite colectarea comparabilă a metricilor pentru modele cu grade diferite de complexitate și pentru trei implementări distincte. Această organizare reduce riscul benchmarkurilor ad-hoc și oferă baza pentru o evaluare empirică completă.

Capitolul 7

Aplicația Web E-Medicine

7.1 Rolul aplicației

Aplicația web este interfața prin care modelul matematic și infrastructura de execuție devin utilizabile. Ea nu este un sistem clinic complet, ci un mediu de lucru pentru încărcarea datelor, configurarea simulărilor SIDARTH sau SIR și compararea rezultatelor obținute în R, Julia și Python. Din această perspectivă, aplicația are rol de instrument experimental.

Fluxul principal începe cu autentificarea utilizatorului. După autentificare, acesta poate încărca un fișier CSV cu date epidemiologice, poate alege modelul, poate introduce parametrii și poate urmări statusul execuției. Când runner-ul finalizează job-ul, interfața afișează traiectoriile compartimentale și permite exportul rezultatelor în format CSV. O pagină separată permite compararea mai multor rulări finalizate, iar pagina de benchmark prezintă agregările calculate la nivel de limbaj.

7.2 Backend Go

Backend-ul este implementat în Go și expune un API REST sub prefixul `/api`. Serverul folosește routerul `chi`, middleware pentru logging, recuperare după panică, timeout de request și CORS, iar accesul la rutele protejate este controlat prin JWT. Alegerea Go este potrivită pentru această componentă deoarece API-ul este stateless, are nevoie de latențe predictibile și poate fi distribuit ușor sub formă de binar static în container.

Structura codului separă intrarea HTTP de accesul la date. Pachetul `handlers` parsează cererile, validează input-ul și construiește răspunsurile JSON. Pachetul `repository` conține interogările SQL și operațiile tranzacționale peste `pgx`. Pachetul `runner` definește schema mesajului trimis în Redis și publicarea job-urilor. Contractul de simulare include tipul modelului, parametrii, condițiile inițiale și conexiunea la baza de date.

7.3 Gestionarea dataseturilor

Dataseturile sunt încărcate printr-un formular multipart. API-ul acceptă câmpurile `name`, `disease_code` și `file`; fișierul trebuie să conțină cel puțin coloanele `date`, `region`, `infected` și `population`. Coloanele `recovered` și `susceptible` sunt opționale, dar atunci când sunt prezente sunt folosite pentru condițiile inițiale. Pentru SIDARTH, cazurile active sunt plasate inițial în compartimentul I, iar persoanele ieșite din infecție în H.

Persistarea datasetului se face într-o tranzacție. Mai întâi este creat rândul din `datasets`, apoi înregistrările epidemiologice sunt inserate prin `CopyFrom`. Această alegere reduce costul încărcării CSV-urilor față de inserarea rând-cu-rând și păstrează atomicitatea: dacă una dintre înregistrări nu poate fi scrisă, datasetul nu rămâne într-o stare parțială.

7.4 Execuția simulărilor

Crearea unei simulări este un proces scurt în API și un proces asincron în ansamblul sistemului. Endpointul `POST /api/simulations` verifică existența datasetului, modelul, limbajul selectat, parametrii și $t_{\max}$, apoi extrage prima înregistrare epidemiologică pentru a determina valorile inițiale. După ce rândul din `simulation_runs` este creat cu status `pending`, API-ul publică mesajul JSON în coada Redis corespunzătoare limbajului.

Runner-ele sunt responsabile pentru tranzițiile ulterioare. Ele marchează rularea ca `running`, execută solver-ul pentru `model_type`, inserează rezultatele și benchmarkul, apoi marchează rularea ca `completed`. Dacă apare o eroare, statusul devine `failed`, iar mesajul este salvat în baza de date. Interfața web nu trebuie să țină o conexiune deschisă pe durata calculului; pagina de rezultat interoghează periodic endpointul de status până la finalizare.

7.5 Frontend Vue.js

Frontend-ul este o aplicație Vue 3 construită cu Vite și TypeScript. Rutarea este gestionată cu Vue Router, iar starea aplicației cu Pinia. Clientul HTTP este o instanță Axios configurată cu prefixul `/api`; un interceptor adaugă automat headerul `Authorization` atunci când există token în `localStorage`.

Paginile sunt organizate după fluxurile principale ale aplicației: `DashboardPage` sumarizează dataseturile și simulările recente, `DatasetsPage` gestionează încărcarea CSV-urilor, `NewSimulationPage` colectează modelul și parametrii, `SimulationResultPage` afișează statusul și rezultatele, `ComparePage` suprapune seriile mai multor rulări, iar

`BenchmarksPage` afișează agregările pe model și limbaj. Componenta `CompartmentChart` folosește Chart.js prin `vue-chartjs` și construiește dinamic seriile în funcție de cheile disponibile în `compartments`. Astfel, același grafic poate afișa atât SIDARTH, cât și SIR.

Decizia de a folosi polling la două secunde este simplă, dar adecvată pentru durata așteptată a simulărilor. O soluție bazată pe WebSocket sau Server-Sent Events ar reduce numărul de cereri, însă ar introduce stare suplimentară în backend. Pentru un API stateless și pentru job-uri care durează de la câteva secunde la câteva minute, polling-ul păstrează implementarea previzibilă.

7.6 Validare funcțională

Validarea funcțională a aplicației poate fi făcută prin fluxuri end-to-end care urmează operațiile reale ale utilizatorului: înregistrare, autentificare, încărcare dataset, pornire simulare SIDARTH, așteptare finalizare, afișare grafic și export CSV. Aceeași suită ar trebui să verifice și rularea SIR, deoarece aceasta asigură compatibilitatea cu modelul secundar și cu comparațiile simple.

Capitolul 8

Concluzii și Direcții de Dezvoltare Viitoare

8.1 Sinteza lucrării

Lucrarea a fost reorientată în jurul modelului SIDARTH, deoarece acesta descrie mai bine situațiile în care diagnosticul, simptomatologia și severitatea clinică influențează evoluția observată a unei epidemii. Modelul SIR rămâne prezent, dar rolul său este secundar: el oferă un reper matematic și numeric simplu, util pentru comparații și verificări de regresie.

Pe această bază a fost construit un sistem software în care datele, simulările și metricile de performanță sunt tratate unitar. API-ul Go oferă autentificare, încărcare de dataseturi și pornire de simulări; Redis decuplează cererile utilizatorului de execuția numerică; runner-ele R, Julia și Python rezolvă modelul cerut prin `model_type`; interfața Vue.js permite urmărirea, vizualizarea și compararea rulărilor. Contribuția practică a sistemului constă în această legătură între model, execuție și observabilitatea rezultatelor.

8.2 Concluzii privind modelele

SIDARTH introduce o structură mai apropiată de realitatea raportării epidemiologice decât SIR. Separarea cazurilor nedepistate de cele diagnosticate și a formelor ușoare de cele severe permite formularea unor întrebări care nu pot fi abordate natural într-un model cu trei compartimente. În același timp, această granularitate crește numărul de parametri și face calibrarea mai dificilă.

SIR rămâne util tocmai prin simplitate. El permite verificarea conservării populației, a pozitivității soluției și a comportamentului solverelor într-un cadru ușor de controlat. În sistemul implementat, SIR este tratat ca model secundar, nu ca explicație completă

a datelor reale.

8.3 Concluzii privind implementarea

Implementarea arată că același contract de simulare poate fi rulat în ecosisteme diferite fără a schimba API-ul sau interfața web. R oferă o formulare compactă prin `deSolve`; Python oferă integrare directă cu infrastructura și bibliotecile științifice uzuale; Julia este potrivită pentru experimente numerice mai ambițioase, mai ales atunci când costul de compilare este amortizat.

Schimbarea schemei rezultatelor din coloane fixe `s/i/r` într-un obiect JSONB `compartments` este una dintre modificările importante ale pivotării. Ea face sistemul mai flexibil și permite adăugarea unor modele noi fără migrații ale tabelei de rezultate. Costul acestei flexibilități este o serializare mai generală și o nevoie mai mare de validare la nivel de aplicație.

8.4 Limitări

Prima limitare este calibrarea parametrilor SIDARTH. Datele OWID folosite în lucrare nu oferă direct distribuția pe compartimentele D, A, R și T. Inițializarea folosită de sistem este reproductibilă, dar nu înlocuiește o estimare epidemiologică dedicată. Pentru rezultate interpretate clinic, modelul trebuie calibrat pe date despre testare, spitalizare, terapie intensivă și vindecări.

A doua limitare privește evaluarea empirică. Infrastructura de benchmark este implementată, dar concluziile cantitative necesită rulări repetate pe configurații hardware documentate. Fără aceste campanii complete, analiza performanței rămâne un cadru de măsurare și nu o demonstrație statistică a superiorității unui limbaj.

O altă limitare este nivelul de maturitate al aplicației web. Fluxurile principale sunt acoperite, însă lipsesc încă teste automate end-to-end, mecanisme de retry mai sofisticate pentru job-uri, politici de rate limiting și audit al operațiilor utilizatorilor. Aceste elemente ar fi necesare înainte de folosirea sistemului într-un context operațional.

8.5 Direcții viitoare

Direcția imediată este calibrarea modelului SIDARTH pe date mai bogate decât seria de cazuri active. O extensie naturală este reintroducerea compartimentului E din SIDARTHE, pentru a modela explicit decesele, și adăugarea unor variațiuni precum SEIR sau SIRS în același contract de simulare.

La nivel de sistem, următorul pas este completarea campaniilor de benchmark și adăugarea unei suite automate de teste. Pentru backend, aceasta înseamnă teste unitare

și de integrare cu o bază de date izolată; pentru frontend, teste care verifică fluxurile reale din browser. În zona de infrastructură, ar fi utilă introducerea unor mecanisme explicite de retry și dead-letter queue pentru job-urile care eșuează repetat.

Pe termen mai lung, sistemul poate fi extins cu vizualizări geografice, surse de date actualizate periodic și module de alertare. Aceste extensii trebuie construite pe aceeași disciplină metodologică: ipotezele modelului, calitatea datelor și metricile software trebuie să rămână vizibile.

Bibliografie

[1] Roy M. Anderson and Robert M. May. *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press, Oxford, 1991.

[2] Herbert W. Hethcote. The mathematics of infectious diseases. *SIAM Review*, 42(4):599–653, 2000.

[3] W. O. Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927.

[4] Odo Diekmann, Johan A. P. Heesterbeek, and Johan A. J. Metz. On the definition and the computation of the basic reproduction ratio $\mathcal{R}_0$ in models for infectious diseases in heterogeneous populations. *Journal of Mathematical Biology*, 28(4):365–382, 1990.

[5] Giulia Giordano, Franco Blanchini, Raffaele Bruno, Patrizio Colaneri, Alessandro Di Filippo, Angela Di Matteo, and Marta Colaneri. Modelling the COVID-19 epidemic and implementation of population-wide interventions in italy. *Nature Medicine*, 26:855–860, 2020.

[6] Pauline van den Driessche and James Watmough. Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission. *Mathematical Biosciences*, 180(1–2):29–48, 2002.

[7] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2 edition, 1993.

[8] John R. Dormand and Peter J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[9] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024.

[10] Karline Soetaert, Thomas Petzoldt, and R. Woodrow Setzer. Solving differential equations in R: Package deSolve. *Journal of Statistical Software*, 33(9):1–25, 2010.

[11] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[12] Christopher Rackauckas and Qing Nie. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017.

[13] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.

[14] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[15] Yugabyte, Inc. YugabyteDB architecture documentation. Documentație tehnică, 2024.

[16] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016.

[17] The Kubernetes Authors. Kubernetes documentation. Documentație tehnică, 2024.

[18] The Go Authors. The Go programming language specification. Specificație oficială, 2024.

[19] Evan You et al. Vue.js documentation. Documentație oficială, 2024.

[20] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC'14)*, pages 305–319, Philadelphia, PA, 2014. USENIX Association.

[21] Rakyll and contributors. Standard Go project layout. Depozit GitHub, 2023.

[22] Michael Jones, John Bradley, and Nat Sakimura. RFC 7519: JSON web token (JWT). Request for Comments, Internet Engineering Task Force, 2015.

[23] Evan You and contributors. Pinia — the intuitive store for Vue.js. Documentație oficială, 2024.

[24] Yugabyte, Inc. YugabyteDB Helm chart for Kubernetes. Depozit GitHub, 2024.

[25] Edouard Mathieu, Hannah Ritchie, Lucas Rodés-Guirao, Cameron Appel, Charlie Giattino, Joe Hasell, Bobbie Macdonald, Saloni Dattani, Diana Beltekian, Esteban Ortiz-Ospina, and Max Roser. Coronavirus pandemic (COVID-19). Our World in Data, 2020.

Anexa A

Cod Sursă — Implementarea R

Această anexă include runner-ul R folosit de sistem pentru consumul job-urilor Redis, rezolvarea modelelor SIDARTH și SIR și persistarea rezultatelor în YugabyteDB.

```r
## Epidemiological model runner -- R.
##
## Consumes simulation jobs from Redis list ``epimodel:jobs:r
    `` (BRPOP via redux),
## solves SIDARTH or SIR ODE systems with ``deSolve::ode`` (
    RK4, fixed step h = 1),
## and inserts compartment trajectories into YugabyteDB.

suppressPackageStartupMessages({
  library(deSolve)
  library(DBI)
  library(RPostgres)
  library(redux)
  library(jsonlite)
})

QUEUE <- "epimodel:jobs:r"
LANGUAGE <- "r"
BRPOP_TIMEOUT_SEC <- 5L
ERROR_BACKOFF_SEC <- 2L

# graceful shutdown flag
.running <- TRUE
handle_signal <- function() {
  message("signal received, shutting down")
  .running <<- FALSE
}
```

```r
# best effort: R has limited signal handling; rely on docker stop SIGTERM = exit

sir_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

sidarth_rhs <- function(t, state, parms) {
  invisible(t)
  with(as.list(c(state, parms)), {
    force <- (alpha * S_I + beta * D + gamma * A + delta * R) / N
    new_infections <- S * force
    dS <- -new_infections
    dI <- new_infections - (epsilon + zeta + lambda) * S_I
    dD <- epsilon * S_I - (eta + rho) * D
    dA <- zeta * S_I - (theta + mu + kappa) * A
    dR <- eta * D + theta * A - (nu + xi) * R
    dT <- mu * A + nu * R - sigma * T
    dH <- lambda * S_I + rho * D + kappa * A + xi * R + sigma * T
    list(c(dS, dI, dD, dA, dR, dT, dH))
  })
}

model_layout <- function(model_type) {
  if (identical(model_type, "sir")) {
    return(list(
      names = c("S", "I", "R"),
      json_names = c("s", "i", "r"),
      rhs = sir_rhs
    ))
  }
  if (identical(model_type, "sidarth")) {
    return(list(
```

```
      names = c("S", "S_I", "D", "A", "R", "T", "H"),
      json_names = c("s", "i", "d", "a", "r", "t", "h"),
      rhs = sidarth_rhs
    ))
  }
  stop(sprintf("unsupported model_type: %s", model_type))
}

peak_rss_mb <- function() {
  # /proc/self/status VmHWM is Linux-only; on macOS fall back to gc()
  status <- tryCatch(readLines("/proc/self/status", warn = FALSE), error = function(e) NULL)
  if (!is.null(status)) {
    line <- grep("^VmHWM:", status, value = TRUE)
    if (length(line) == 1) {
      kb <- as.numeric(sub("[^0-9]+([0-9]+).*", "\\1", line))
      return(kb / 1024)
    }
  }
  used_mb <- sum(gc()[, 2])
  used_mb
}

parse_dsn <- function(dsn) {
  # postgres://user:pass@host:port/db?sslmode=disable
  m <- regmatches(dsn, regexec(
    "postgres(?:ql)?://([^:]+):([^@]*)@([^:/]+):([0-9]+)/([^?]+)", dsn
  ))[[1]]
  if (length(m) < 6) stop(sprintf("invalid DSN: %s", dsn))
  list(user = m[2], password = m[3], host = m[4],
       port = as.integer(m[5]), dbname = m[6])
}

db_connect <- function(dsn) {
  p <- parse_dsn(dsn)
  dbConnect(RPostgres::Postgres(),
            host = p$host, port = p$port, dbname = p$dbname,
            user = p$user, password = p$password)
}
```

```r

run_simulation <- function(conn, job) {
  sim_id <- job$simulation_id
  model_type <- if (!is.null(job$model_type)) job$model_type
   else "sidarth"
  layout <- model_layout(model_type)
  parameters <- as.list(job$parameters)
  parameters <- lapply(parameters, as.numeric)
  N <- as.numeric(job$population_n)
  t_max <- as.integer(job$t_max)
  initials <- as.list(job$initial)
  y0 <- setNames(
    vapply(layout$json_names, function(name) {
      value <- initials[[name]]
      if (is.null(value)) 0 else as.numeric(value)
    }, numeric(1)),
    layout$names
  )

  dbExecute(conn, "UPDATE simulation_runs SET status = '
   running' WHERE id = $1",
            params = list(sim_id))

  rss_before <- peak_rss_mb()
  wall_start <- proc.time()["elapsed"]
  cpu_start  <- proc.time()["user.self"] + proc.time()["sys.
   self"]

  times <- seq(0, t_max, by = 1)
  parms <- c(unlist(parameters), N = N)
  out <- ode(y = y0, times = times, func = layout$rhs, parms =
    parms, method = "rk4")

  wall_ms <- as.integer((proc.time()["elapsed"] - wall_start)
   * 1000)
  cpu_ms  <- as.integer(((proc.time()["user.self"] + proc.time
   ()["sys.self"]) - cpu_start) * 1000)
  peak_mb <- max(peak_rss_mb(), rss_before)

  times_out <- as.integer(out[, "time"])

```

```
  dbWithTransaction(conn, {
    for (idx in seq_along(times_out)) {
      values <- as.list(pmax(0, as.numeric(out[idx, layout$
   names])))
      names(values) <- layout$json_names
      dbExecute(conn,
        "INSERT INTO simulation_results (simulation_id, t,
   compartments) VALUES ($1, $2, $3::jsonb)",
        params = list(sim_id, times_out[idx], toJSON(values,
   auto_unbox = TRUE)))
    }
    dbExecute(conn,
      "INSERT INTO benchmarks (simulation_id, wall_time_ms,
   cpu_time_ms, peak_memory_mb) VALUES ($1, $2, $3, $4)",
      params = list(sim_id, wall_ms, cpu_ms, peak_mb))
    dbExecute(conn,
      "UPDATE simulation_runs SET status = 'completed',
   completed_at = NOW() WHERE id = $1",
      params = list(sim_id))
  })

  message(sprintf("sim %s done: wall=%dms cpu=%dms mem=%.1fMB
   rows=%d",
                  sim_id, wall_ms, cpu_ms, peak_mb, length(
   times_out)))
}

mark_failed <- function(conn, sim_id, msg) {
  tryCatch(
    dbExecute(conn,
      "UPDATE simulation_runs SET status = 'failed', error_
   message = $1, completed_at = NOW() WHERE id = $2",
      params = list(substr(msg, 1, 2000), sim_id)),
    error = function(e) message(sprintf("could not mark failed
   : %s", e$message))
  )
}

main <- function() {
  redis_url <- Sys.getenv("REDIS_URL", "redis://redis:6379/0")
  db_dsn_env <- Sys.getenv("DATABASE_URL", "")
```

```
169
170 # redux uses host/port/db number; parse URL
171 m <- regmatches(redis_url, regexec(
172   "redis://(?:[^@]*@)?([^:/]+):([0-9]+)(?:/([0-9]+))?",
    redis_url
173 ))[[1]]
174 if (length(m) < 4) stop(sprintf("invalid REDIS_URL: %s",
    redis_url))
175 rdb <- redux::hiredis(host = m[2], port = as.integer(m[3]),
176                       db = if (m[4] == "") 0L else as.
    integer(m[4]))
177 rdb$PING()
178 message(sprintf("runner-r connected to redis, consuming %s",
    QUEUE))
179
180 while (.running) {
181   item <- tryCatch(rdb$BRPOP(QUEUE, BRPOP_TIMEOUT_SEC),
182                    error = function(e) {
183                      message(sprintf("redis error: %s", e$
    message))
184                      Sys.sleep(ERROR_BACKOFF_SEC)
185                      NULL
186                    })
187   if (is.null(item)) next
188
189   raw <- item[[2]]
190   job <- tryCatch(fromJSON(raw), error = function(e) {
191     message(sprintf("invalid job json: %s", e$message))
192     NULL
193   })
194   if (is.null(job)) next
195
196   sim_id <- if (!is.null(job$simulation_id)) job$simulation_
    id else "?"
197   dsn <- if (!is.null(job$db_dsn) && nzchar(job$db_dsn)) job
    $db_dsn else db_dsn_env
198   if (!nzchar(dsn)) {
199     message(sprintf("sim %s: no db_dsn", sim_id))
200     next
201   }
202
```

```
203     conn <- tryCatch(db_connect(dsn), error = function(e) {
204       message(sprintf("sim %s db connect failed: %s", sim_id,
    e$message))
205       NULL
206     })
207     if (is.null(conn)) next
208
209     tryCatch({
210       run_simulation(conn, job)
211     }, error = function(e) {
212       message(sprintf("sim %s failed: %s", sim_id, e$message))
213       mark_failed(conn, sim_id, e$message)
214     }, finally = {
215       dbDisconnect(conn)
216     })
217   }
218   message("runner-r stopped")
219 }
220
221 main()
```

Listing A.1: Runner-ul R pentru modelele SIDARTH și SIR.

Anexa B

Cod Sursă — Implementarea Julia

Această anexă include runner-ul Julia, bazat pe `DifferentialEquations.jl`, `Redis.jl` și `LibPQ.jl`.

```
#=
Epidemiological model runner -- Julia.

Consumes simulation jobs from Redis list 'epimodel:jobs:julia'
    (BRPOP),
solves SIDARTH or SIR ODE systems with DifferentialEquations.
   jl (Tsit5, saveat=1),
and inserts compartment trajectories into YugabyteDB via LibPQ
   .
=#

using DifferentialEquations
using JSON3
using LibPQ
using Redis
using Dates

const QUEUE = "epimodel:jobs:julia"
const LANGUAGE = "julia"
const BRPOP_TIMEOUT_SEC = 5
const ERROR_BACKOFF_SEC = 2

mutable struct RunCtx
    running::Bool
end
const CTX = RunCtx(true)

```

```
function sir_rhs!(du, u, p, t)
    _ = t
    S, I, R = u
    β = p["beta"]
    γ = p["gamma"]
    N = p["N"]
    du[1] = -β * S * I / N
    du[2] =  β * S * I / N - γ * I
    du[3] =  γ * I
end

function sidarth_rhs!(du, u, p, t)
    _ = t
    S, I, D, A, R, T, H = u
    N = p["N"]
    force = (p["alpha"] * I + p["beta"] * D + p["gamma"] * A + p["delta"] * R) / N
    new_infections = S * force

    du[1] = -new_infections
    du[2] = new_infections - (p["epsilon"] + p["zeta"] + p["lambda"]) * I
    du[3] = p["epsilon"] * I - (p["eta"] + p["rho"]) * D
    du[4] = p["zeta"] * I - (p["theta"] + p["mu"] + p["kappa"]) * A
    du[5] = p["eta"] * D + p["theta"] * A - (p["nu"] + p["xi"]) * R
    du[6] = p["mu"] * A + p["nu"] * R - p["sigma"] * T
    du[7] = p["lambda"] * I + p["rho"] * D + p["kappa"] * A + p["xi"] * R + p["sigma"] * T
    nothing
end

function model_layout(model_type::AbstractString)
    if model_type == "sir"
        return (state_names = ["s", "i", "r"], rhs = sir_rhs!)
    elseif model_type == "sidarth"
        return (state_names = ["s", "i", "d", "a", "r", "t", "h"], rhs = sidarth_rhs!)
    end
    error("unsupported model_type: $model_type")
```

```
60 end
61
62 peak_rss_mb() = Sys.maxrss() / (1024 * 1024)
63
64 function parse_redis_url(url::AbstractString)
65     m = match(r"^redis://(?:[^@]*@)?(?<host>[^:/]+):(?<port>\d
   +)(?:/(?<db>\d+))?$", url)
66     m === nothing && error("invalid REDIS_URL: $url")
67     (host = m[:host],
68      port = parse(Int, m[:port]),
69      db   = m[:db] === nothing ? 0 : parse(Int, m[:db]))
70 end
71
72 function run_simulation(conn::LibPQ.Connection, job::Dict{
   Symbol, Any})
73     sim_id = String(job[:simulation_id])
74     model_type = String(get(job, :model_type, "sidarth"))
75     layout = model_layout(model_type)
76     params = Dict{String, Float64}(String(k) => Float64(v) for
     (k, v) in pairs(job[:parameters]))
77     N = Float64(job[:population_n])
78     t_max = Int(job[:t_max])
79     params["N"] = N
80     initial = Dict{String, Float64}(String(k) => Float64(v)
   for (k, v) in pairs(job[:initial]))
81     y0 = Float64[get(initial, name, 0.0) for name in layout.
   state_names]
82
83     LibPQ.execute(conn,
84         "UPDATE simulation_runs SET status = 'running' WHERE
   id = \$1",
85         [sim_id])
86
87     rss_before = peak_rss_mb()
88     wall_start = time_ns()
89     cpu_start  = time()  # process CPU not directly exposed;
   use elapsed as fallback
90
91     prob = ODEProblem(layout.rhs, y0, (0.0, Float64(t_max)),
   params)
92     sol = solve(prob, Tsit5(); saveat = 0:1.0:t_max, reltol =
```

```
    1e-8, abstol = 1e-10)

     wall_ms = round(Int, (time_ns() - wall_start) / 1_000_000)
     cpu_ms  = round(Int, (time() - cpu_start) * 1000)
     peak_mb = max(peak_rss_mb(), rss_before)

     LibPQ.execute(conn, "BEGIN")
     try
         stmt = LibPQ.prepare(conn,
             "INSERT INTO simulation_results (simulation_id, t,
     compartments) VALUES (\$1, \$2, \$3::jsonb)")
         for (idx, t) in enumerate(sol.t)
             u = sol.u[idx]
             values = Dict(name => max(0.0, Float64(u[pos]))
    for (pos, name) in enumerate(layout.state_names))
             LibPQ.execute(stmt, (sim_id, Int(t), JSON3.write(
    values)))
         end
         LibPQ.execute(conn,
             "INSERT INTO benchmarks (simulation_id,
    wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES (\$1, \$2
    , \$3, \$4)",
             [sim_id, wall_ms, cpu_ms, peak_mb])
         LibPQ.execute(conn,
             "UPDATE simulation_runs SET status = 'completed',
    completed_at = NOW() WHERE id = \$1",
             [sim_id])
         LibPQ.execute(conn, "COMMIT")
     catch e
         LibPQ.execute(conn, "ROLLBACK")
         rethrow(e)
     end

     println("sim $sim_id done: wall=$(wall_ms)ms cpu=$(cpu_ms)
    ms mem=$(round(peak_mb, digits=1))MB rows=$(length(sol.t))"
    )
end

function mark_failed(conn::LibPQ.Connection, sim_id::
    AbstractString, msg::AbstractString)
     try
```

```
124         LibPQ.execute(conn,
125             "UPDATE simulation_runs SET status = 'failed',
    error_message = \$1, completed_at = NOW() WHERE id = \$2",
126             [first(msg, 2000), sim_id])
127     catch e
128         @warn "could not mark failed" exception=e
129     end
130 end
131
132 function main()
133     redis_url = get(ENV, "REDIS_URL", "redis://redis:6379/0")
134     db_dsn_env = get(ENV, "DATABASE_URL", "")
135     rinfo = parse_redis_url(redis_url)
136     rcli = RedisConnection(host = rinfo.host, port = rinfo.
    port, db = rinfo.db)
137     Redis.ping(rcli)
138     println("runner-julia connected to redis, consuming $QUEUE
    ")
139
140     while CTX.running
141         item = try
142             Redis.brpop(rcli, [QUEUE], BRPOP_TIMEOUT_SEC)
143         catch e
144             @warn "redis error" exception=e
145             sleep(ERROR_BACKOFF_SEC)
146             nothing
147         end
148         item === nothing && continue
149         # Redis.brpop returns Vector{String} [queue_name,
    payload] or nil
150         length(item) < 2 && continue
151         raw = item[2]
152
153         job = try
154             JSON3.read(raw, Dict{Symbol, Any})
155         catch e
156             @warn "invalid job json" exception=e
157             continue
158         end
159
160         sim_id = get(job, :simulation_id, "?")
```

```
161         dsn = get(job, :db_dsn, "")
162         if isempty(dsn)
163             dsn = db_dsn_env
164         end
165         if isempty(dsn)
166             @warn "no db_dsn for sim" sim_id
167             continue
168         end
169 
170         conn = try
171             LibPQ.Connection(dsn)
172         catch e
173             @warn "db connect failed" sim_id exception=e
174             continue
175         end
176         try
177             run_simulation(conn, job)
178         catch e
179             @warn "sim failed" sim_id exception=e
180             mark_failed(conn, String(sim_id), sprint(showerror
    , e))
181         finally
182             close(conn)
183         end
184     end
185     println("runner-julia stopped")
186 end
187 
188 main()
```

Listing B.1: Runner-ul Julia pentru modelele SIDARTH și SIR.

Anexa C

Cod Sursă — Implementarea Python

Această anexă include runner-ul Python, care folosește `scipy.integrate.solve_ivp` pentru integrarea numerică și `psycopg` pentru scrierea rezultatelor.

```
"""Epidemiological model runner -- Python.

Consumes simulation jobs from Redis list ``epimodel:jobs:python`` (BRPOP),
solves SIDARTH or SIR ODE systems with ``scipy.integrate.solve_ivp`` (RK45),
inserts compartment trajectories into YugabyteDB and records the benchmark
(wall time, CPU time, peak RSS) into ``benchmarks``.
"""

from __future__ import annotations

import json
import os
import resource
import signal
import sys
import time
from typing import Any

import numpy as np
import psycopg
import redis
from psycopg.rows import dict_row
from psycopg.types.json import Json
from scipy.integrate import solve_ivp
```

```
25
26 QUEUE = "epimodel:jobs:python"
27 LANGUAGE = "python"
28 BRPOP_TIMEOUT_SEC = 5
29 ERROR_BACKOFF_SEC = 2
30
31 _running = True
32
33
34 def _handle_signal(signum, _frame):
35     global _running
36     print(f"signal {signum} received, shutting down", file=sys
   .stderr, flush=True)
37     _running = False
38
39
40 def sir_rhs(t: float, y: np.ndarray, params: dict[str, float],
    n: float) -> list[float]:
41     del t
42     s, i, r = y
43     beta = params["beta"]
44     gamma = params["gamma"]
45     return [
46         -beta * s * i / n,
47         beta * s * i / n - gamma * i,
48         gamma * i,
49     ]
50
51
52 def sidarth_rhs(t: float, y: np.ndarray, params: dict[str,
   float], n: float) -> list[float]:
53     del t
54     s, i, d, a, r, threatened, h = y
55     force = (params["alpha"] * i + params["beta"] * d + params
   ["gamma"] * a + params["delta"] * r) / n
56     new_infections = s * force
57
58     return [
59         -new_infections,
60         new_infections - (params["epsilon"] + params["zeta"] +
    params["lambda"]) * i,
```

```
        params["epsilon"] * i - (params["eta"] + params["rho"]) * d,
        params["zeta"] * i - (params["theta"] + params["mu"] + params["kappa"]) * a,
        params["eta"] * d + params["theta"] * a - (params["nu"] + params["xi"]) * r,
        params["mu"] * a + params["nu"] * r - params["sigma"] * threatened,
        params["lambda"] * i + params["rho"] * d + params["kappa"] * a + params["xi"] * r + params["sigma"] * threatened,
    ]


def model_layout(model_type: str) -> tuple[list[str], Any]:
    if model_type == "sir":
        return ["s", "i", "r"], sir_rhs
    if model_type == "sidarth":
        return ["s", "i", "d", "a", "r", "t", "h"], sidarth_rhs
    raise ValueError(f"unsupported model_type: {model_type}")


def _peak_rss_mb() -> float:
    """Return peak RSS in MB across platforms.

    Linux returns ``ru_maxrss`` in KiB, macOS in bytes.
    """
    rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
    if sys.platform == "darwin":
        return rss / (1024 * 1024)
    return rss / 1024


def run_simulation(conn: psycopg.Connection, job: dict[str, Any]) -> None:
    sim_id = job["simulation_id"]
    model_type = job.get("model_type", "sidarth")
    params = {str(k): float(v) for k, v in job["parameters"].items()}
    initial = {str(k): float(v) for k, v in job["initial"].
```

```
    items()}
     n = float(job["population_n"])
     t_max = int(job["t_max"])
     compartments, rhs = model_layout(model_type)
     y0 = [initial.get(name, 0.0) for name in compartments]

     with conn.transaction():
         conn.execute(
             "UPDATE simulation_runs SET status = 'running'
WHERE id = %s",
             (sim_id,),
         )

     rss_before = _peak_rss_mb()
     wall_start = time.perf_counter()
     cpu_start = time.process_time()

     sol = solve_ivp(
         fun=rhs,
         t_span=(0.0, float(t_max)),
         y0=y0,
         method="RK45",
         t_eval=np.arange(0, t_max + 1, dtype=float),
         args=(params, n),
         rtol=1e-8,
         atol=1e-10,
     )
     if not sol.success:
         raise RuntimeError(f"solve_ivp failed: {sol.message}")

     wall_ms = int((time.perf_counter() - wall_start) * 1000)
     cpu_ms = int((time.process_time() - cpu_start) * 1000)
     peak_mb = max(_peak_rss_mb(), rss_before)

     rows = []
     for idx, t in enumerate(sol.t):
         values = {
             name: max(0.0, float(sol.y[pos][idx]))
             for pos, name in enumerate(compartments)
         }
         rows.append((sim_id, int(t), Json(values)))
```

```
132
133     with conn.transaction():
134         with conn.cursor() as cur:
135             cur.executemany(
136                 "INSERT INTO simulation_results (simulation_id
    , t, compartments) VALUES (%s, %s, %s)",
137                 rows,
138             )
139             cur.execute(
140                 "INSERT INTO benchmarks (simulation_id,
    wall_time_ms, cpu_time_ms, peak_memory_mb) VALUES (%s, %s,
    %s, %s)",
141                 (sim_id, wall_ms, cpu_ms, peak_mb),
142             )
143             cur.execute(
144                 "UPDATE simulation_runs SET status = '
    completed', completed_at = NOW() WHERE id = %s",
145                 (sim_id,),
146             )
147     print(
148         f"sim {sim_id} done: wall={wall_ms}ms cpu={cpu_ms}ms
    mem={peak_mb:.1f}MB rows={len(rows)}",
149         flush=True,
150     )
151
152
153 def mark_failed(conn: psycopg.Connection, sim_id: str, msg:
    str) -> None:
154     try:
155         with conn.transaction():
156             conn.execute(
157                 "UPDATE simulation_runs SET status = 'failed',
     error_message = %s, completed_at = NOW() WHERE id = %s",
158                 (msg[:2000], sim_id),
159             )
160     except Exception as exc:  # noqa: BLE001
161         print(f"could not mark failed: {exc}", file=sys.stderr
    , flush=True)
162
163
164 def main() -> int:
```

```
    signal.signal(signal.SIGINT, _handle_signal)
    signal.signal(signal.SIGTERM, _handle_signal)

    redis_url = os.environ["REDIS_URL"]
    db_dsn = os.environ.get("DATABASE_URL")

    rdb = redis.from_url(redis_url)
    rdb.ping()
    print(f"runner-python connected to redis, consuming {QUEUE}", flush=True)

    while _running:
        try:
            item = rdb.brpop(QUEUE, timeout=BRPOP_TIMEOUT_SEC)
        except redis.RedisError as exc:
            print(f"redis error: {exc}", file=sys.stderr, flush=True)
            time.sleep(ERROR_BACKOFF_SEC)
            continue
        if item is None:
            continue

        _, raw = item
        try:
            job = json.loads(raw)
        except json.JSONDecodeError as exc:
            print(f"invalid job json: {exc}", file=sys.stderr, flush=True)
            continue

        sim_id = job.get("simulation_id", "?")
        dsn = job.get("db_dsn") or db_dsn
        if not dsn:
            print(f"sim {sim_id}: no db_dsn", file=sys.stderr, flush=True)
            continue

        try:
            with psycopg.connect(dsn, row_factory=dict_row) as conn:
                run_simulation(conn, job)
```

```
201         except Exception as exc:  # noqa: BLE001
202             print(f"sim {sim_id} failed: {exc}", file=sys.
    stderr, flush=True)
203             try:
204                 with psycopg.connect(dsn) as conn:
205                     mark_failed(conn, sim_id, str(exc))
206             except Exception as inner:  # noqa: BLE001
207                 print(f"could not connect to mark failed: {
    inner}", file=sys.stderr, flush=True)
208 
209     print("runner-python stopped", flush=True)
210     return 0
211 
212 
213 if __name__ == "__main__":
214     sys.exit(main())
```

Listing C.1: Runner-ul Python pentru modelele SIDARTH și SIR.

Anexa D

Schema Bazei de Date

Anexa include migrațiile SQL care definesc schema relațională folosită de API și runnere.

```sql
CREATE TABLE users (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email         VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);
```

Listing D.1: Crearea tabelului `users`.

```sql
CREATE TABLE datasets (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL REFERENCES users(id) ON DELETE
    CASCADE,
    name          VARCHAR(255) NOT NULL,
    disease_code  VARCHAR(64) NOT NULL,
    uploaded_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    record_count  INTEGER NOT NULL DEFAULT 0
);

CREATE INDEX idx_datasets_user_id ON datasets(user_id);
CREATE INDEX idx_datasets_disease_code ON datasets(
   disease_code);
```

Listing D.2: Crearea tabelului `datasets`.

```sql
CREATE TABLE epidemiological_records (
    id            UUID DEFAULT gen_random_uuid(),
```

```
    dataset_id    UUID NOT NULL REFERENCES datasets(id) ON
   DELETE CASCADE,
    recorded_date DATE NOT NULL,
    region_code   VARCHAR(10) NOT NULL,
    susceptible   BIGINT,
    infected      BIGINT NOT NULL,
    recovered     BIGINT,
    population    BIGINT NOT NULL,
    PRIMARY KEY (dataset_id, recorded_date, id),
    CHECK (infected >= 0),
    CHECK (population > 0),
    CHECK (recovered IS NULL OR recovered >= 0),
    CHECK (susceptible IS NULL OR susceptible >= 0)
);

CREATE INDEX idx_epi_records_region ON epidemiological_records
   (region_code);
```

Listing D.3: Crearea tabelului `epidemiological_records`.

```
CREATE TABLE simulation_runs (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id       UUID NOT NULL REFERENCES users(id) ON DELETE
    CASCADE,
    dataset_id    UUID NOT NULL REFERENCES datasets(id) ON
   DELETE CASCADE,
    model_type    VARCHAR(16) NOT NULL DEFAULT 'sidarth',
    language      VARCHAR(16) NOT NULL,
    parameters    JSONB NOT NULL,
    population_n  BIGINT NOT NULL,
    t_max         INTEGER NOT NULL,
    status        VARCHAR(16) NOT NULL DEFAULT 'pending',
    error_message TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at  TIMESTAMPTZ,
    CHECK (model_type IN ('sidarth', 'sir')),
    CHECK (language IN ('r', 'julia', 'python')),
    CHECK (status IN ('pending', 'running', 'completed', '
   failed')),
    CHECK (jsonb_typeof(parameters) = 'object'),
    CHECK (population_n > 0),
    CHECK (t_max > 0)
```

```
);

CREATE INDEX idx_sim_runs_user_id ON simulation_runs(user_id);
CREATE INDEX idx_sim_runs_dataset_id ON simulation_runs(
    dataset_id);
CREATE INDEX idx_sim_runs_status ON simulation_runs(status);
CREATE INDEX idx_sim_runs_language ON simulation_runs(language
    );
CREATE INDEX idx_sim_runs_model_type ON simulation_runs(
    model_type);
```

Listing D.4: Crearea tabelului `simulation_runs`.

```
CREATE TABLE simulation_results (
    simulation_id UUID NOT NULL REFERENCES simulation_runs(id)
    ON DELETE CASCADE,
    t             INTEGER NOT NULL,
    compartments  JSONB NOT NULL,
    PRIMARY KEY (simulation_id, t),
    CHECK (t >= 0),
    CHECK (jsonb_typeof(compartments) = 'object')
);
```

Listing D.5: Crearea tabelului `simulation_results`.

```
CREATE TABLE benchmarks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid()
    ,
    simulation_id   UUID NOT NULL REFERENCES simulation_runs(
    id) ON DELETE CASCADE,
    wall_time_ms    BIGINT NOT NULL,
    cpu_time_ms     BIGINT NOT NULL,
    peak_memory_mb  REAL NOT NULL,
    CHECK (wall_time_ms >= 0),
    CHECK (cpu_time_ms >= 0),
    CHECK (peak_memory_mb >= 0)
);

CREATE INDEX idx_benchmarks_simulation_id ON benchmarks(
    simulation_id);
```

Listing D.6: Crearea tabelului `benchmarks`.

Anexa E

Configurații Kubernetes

Această anexă include manifestele declarative pentru rularea sistemului pe un cluster Kubernetes. Toate resursele sunt aplicate în namespace-ul `epimodel` și urmează structura arhitecturală descrisă în capitolul 4: API stateless, frontend servit prin Nginx, runner-e scalabile independent, Redis pentru cozi de mesaje și YugabyteDB pentru persistență.

Manifestele se află în directorul `code/k8s/` al repository-ului și sunt organizate cu prefixe numerice pentru aplicare secvențială (`kubectl apply -f code/k8s/`). Table E.1 sintetizează conținutul directorului.

Listările care urmează reproduc manifestele cele mai relevante pentru explicitarea arhitecturii: namespace-ul de izolare, stratul de persistență (YugabyteDB tserver), coada de mesaje (Redis), API-ul, un exemplu de runner (Julia, cel mai mare consumator de memorie) și ingress-ul de rutare. Restul manifestelor (master-ul YugabyteDB, ConfigMap-ul, template-ul Secret, Job-ul de migrare, celelalte două runnere, frontend-ul) urmează aceleași convenții și pot fi consultate direct în repository.

Namespace dedicat

```
apiVersion: v1
kind: Namespace
metadata:
  name: epimodel
  labels:
    app.kubernetes.io/part-of: epimodel
    app.kubernetes.io/managed-by: kubectl
```

Listing E.1: `00-namespace.yaml` — Namespace-ul dedicat aplicației.

Tabela E.1: Manifestele Kubernetes incluse în repository.

Fișier	**Resurse principale**	**Rol**
`00-namespace.yaml`	`Namespace`	Izolează aplicația într-un namespace dedicat
`01-configmap.yaml`	`ConfigMap`	Variabile non-sensibile pentru API
`02-secret.example.yaml`	`Secret` (template)	Șablon pentru JWT și DSN-ul bazei de date
`10-yugabyte-master.yaml`	`StatefulSet` (×3)	YugabyteDB master (coordonatorul cluster-ului)
`11-yugabyte-tserver.yaml`	`StatefulSet` (×3)	YugabyteDB tserver (YSQL pe portul 5433)
`20-redis.yaml`	`StatefulSet` (×1)	Coadă Redis cu persistență AOF
`30-migrate-job.yaml`	`Job`	Aplică migrațiile SQL prin `golang-migrate`
`40-api.yaml`	`Deployment` + `Service`	Backend Go (REST API)
`50-runner-python.yaml`	`Deployment`	Runner Python (`scipy.solve_ivp`)
`51-runner-r.yaml`	`Deployment`	Runner R (`deSolve::ode`)
`52-runner-julia.yaml`	`Deployment`	Runner Julia (`DifferentialEquations.jl`)
`60-frontend.yaml`	`Deployment` + `Service`	Frontend Vue.js servit prin Nginx
`70-ingress.yaml`	`Ingress`	Rutare HTTP externă (`/api` și `/`)

YugabyteDB tserver

Tserver-ul rulează ca `StatefulSet` cu trei replici și expune două servicii: unul *headless* pentru descoperirea internă a peer-urilor (necesar protocolului Raft) și unul `ClusterIP` pentru consumatorii de YSQL (API-ul și runner-ele).

```
apiVersion: v1
kind: Service
metadata:
  name: yb-tservers
  namespace: epimodel
  labels:
    app.kubernetes.io/name: yb-tserver
    app.kubernetes.io/part-of: epimodel
spec:
  clusterIP: None
  publishNotReadyAddresses: true
  ports:
    - name: rpc
      port: 9100
      targetPort: 9100
    - name: ui
      port: 9000
      targetPort: 9000
    - name: ysql
      port: 5433
      targetPort: 5433
  selector:
    app.kubernetes.io/name: yb-tserver
---
apiVersion: v1
kind: Service
metadata:
  name: yb-tserver
  namespace: epimodel
  labels:
    app.kubernetes.io/name: yb-tserver
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: ysql
```

```
      port: 5433
      targetPort: 5433
  selector:
    app.kubernetes.io/name: yb-tserver
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: yb-tserver
  namespace: epimodel
  labels:
    app.kubernetes.io/name: yb-tserver
    app.kubernetes.io/part-of: epimodel
spec:
  serviceName: yb-tservers
  replicas: 3
  podManagementPolicy: Parallel
  selector:
    matchLabels:
      app.kubernetes.io/name: yb-tserver
  template:
    metadata:
      labels:
        app.kubernetes.io/name: yb-tserver
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 60
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app.kubernetes.io/name: yb-tserver
                topologyKey: kubernetes.io/hostname
      containers:
        - name: yb-tserver
          image: yugabytedb/yugabyte:2.21.1.0-b271
          imagePullPolicy: IfNotPresent
          env:
```

```
78              - name: POD_IP
79                valueFrom:
80                  fieldRef:
81                    fieldPath: status.podIP
82              - name: POD_NAME
83                valueFrom:
84                  fieldRef:
85                    fieldPath: metadata.name
86              - name: NAMESPACE
87                valueFrom:
88                  fieldRef:
89                    fieldPath: metadata.namespace
90            command:
91              - /bin/bash
92              - -c
93              - >
94                /home/yugabyte/bin/yb-tserver
95                --fs_data_dirs=/mnt/data0
96                --tserver_master_addrs=yb-master-0.yb-masters.${
   NAMESPACE}.svc.cluster.local:7100,yb-master-1.yb-masters.${
   NAMESPACE}.svc.cluster.local:7100,yb-master-2.yb-masters.${
   NAMESPACE}.svc.cluster.local:7100
97                --rpc_bind_addresses=${POD_NAME}.yb-tservers.${
   NAMESPACE}.svc.cluster.local:9100
98                --server_broadcast_addresses=${POD_NAME}.yb-
   tservers.${NAMESPACE}.svc.cluster.local:9100
99                --enable_ysql=true
100               --pgsql_proxy_bind_address=0.0.0.0:5433
101               --cql_proxy_bind_address=0.0.0.0:9042
102               --webserver_interface=0.0.0.0
103               --ysql_num_shards_per_tserver=4
104               --logtostderr
105           ports:
106             - name: rpc
107               containerPort: 9100
108             - name: ui
109               containerPort: 9000
110             - name: ysql
111               containerPort: 5433
112             - name: ycql
113               containerPort: 9042
```

```
          volumeMounts:
            - name: data
              mountPath: /mnt/data0
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
            limits:
              cpu: "2"
              memory: 2Gi
          readinessProbe:
            tcpSocket:
              port: 5433
            initialDelaySeconds: 30
            periodSeconds: 10
            timeoutSeconds: 5
          livenessProbe:
            tcpSocket:
              port: 9100
            initialDelaySeconds: 90
            periodSeconds: 30
            timeoutSeconds: 10
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 50Gi
```

Listing E.2: `11-yugabyte-tserver.yaml` — StatefulSet și servicii pentru YugabyteDB tserver.

Redis

Redis-ul este implementat ca `StatefulSet` cu o singură replică și volum persistent pentru AOF (*append-only file*), pentru ca starea cozilor să supraviețuiască restartărilor pod-ului.

```
apiVersion: v1
kind: Service
```

```
metadata:
  name: redis
  namespace: epimodel
  labels:
    app.kubernetes.io/name: redis
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: redis
      port: 6379
      targetPort: 6379
  selector:
    app.kubernetes.io/name: redis
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  namespace: epimodel
  labels:
    app.kubernetes.io/name: redis
    app.kubernetes.io/part-of: epimodel
spec:
  serviceName: redis
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: redis
  template:
    metadata:
      labels:
        app.kubernetes.io/name: redis
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 30
      containers:
        - name: redis
          image: redis:7-alpine
          imagePullPolicy: IfNotPresent
          args:
```

```
            - redis-server
            - --appendonly
            - "yes"
            - --save
            - "60"
            - "100"
          ports:
            - name: redis
              containerPort: 6379
          volumeMounts:
            - name: data
              mountPath: /data
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 256Mi
          readinessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 3
          livenessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 15
            periodSeconds: 15
            timeoutSeconds: 5
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 5Gi
```

Listing E.3: `20-redis.yaml` — Redis cu persistență AOF.

API Go

API-ul rulează ca `Deployment` cu două replici pentru toleranță la pierderea unui pod. Configurația non-sensibilă vine din `ConfigMap`, iar parolele și secretele din `Secret`. Probele HTTP pe `/api/health` verifică atât pod-ul, cât și conectivitatea efectivă către baza de date și Redis.

```
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: epimodel
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/part-of: epimodel
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 8080
      targetPort: 8080
  selector:
    app.kubernetes.io/name: api
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: epimodel
  labels:
    app.kubernetes.io/name: api
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: api
```

```
  template:
    metadata:
      labels:
        app.kubernetes.io/name: api
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 30
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    app.kubernetes.io/name: api
                topologyKey: kubernetes.io/hostname
      containers:
        - name: api
          image: emedicine/api:latest
          imagePullPolicy: IfNotPresent
          envFrom:
            - configMapRef:
                name: epimodel-config
            - secretRef:
                name: epimodel-secrets
          ports:
            - name: http
              containerPort: 8080
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /api/health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
```

```
            timeoutSeconds: 3
          livenessProbe:
            httpGet:
              path: /api/health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 30
            timeoutSeconds: 5
```

Listing E.4: `40-api.yaml` — Deployment și Service pentru API-ul Go.

Runner Julia

Runner-ul Julia este Deployment-ul cu cele mai generoase limite de resurse (512 MiB request, 2 GiB limit) din cauza costului de compilare JIT al `DifferentialEquations.jl`. Runner-ele R și Python au structuri identice, dar cu limite mai mici.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: runner-julia
  namespace: epimodel
  labels:
    app.kubernetes.io/name: runner-julia
    app.kubernetes.io/component: runner
    app.kubernetes.io/part-of: epimodel
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app.kubernetes.io/name: runner-julia
  template:
    metadata:
      labels:
        app.kubernetes.io/name: runner-julia
        app.kubernetes.io/component: runner
        app.kubernetes.io/part-of: epimodel
    spec:
      terminationGracePeriodSeconds: 60
      containers:
```

```
        - name: runner-julia
          image: emedicine/runner-julia:latest
          imagePullPolicy: IfNotPresent
          env:
            - name: REDIS_URL
              valueFrom:
                configMapKeyRef:
                  name: epimodel-config
                  key: REDIS_URL
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: epimodel-secrets
                  key: DATABASE_URL
          resources:
            requests:
              cpu: 500m
              memory: 512Mi
            limits:
              cpu: "2"
              memory: 2Gi
```

Listing E.5: `52-runner-julia.yaml` — Deployment pentru runner-ul Julia.

Ingress de rutare

Ingress-ul nginx rutează cererile externe către cele două servicii vizibile public: API-ul Go pentru cererile sub prefixul `/api` și frontend-ul Vue.js pentru restul.

```
# Ingress nginx pentru rutarea traficului HTTP extern.
# Presupune că nginx-ingress controller este deja instalat în cluster
# (https://kubernetes.github.io/ingress-nginx/deploy/).
#
# Pentru testare locală, mapează în /etc/hosts:
#   <IP-NODE-INGRESS>  epimodel.local
#
# Pentru producţie, înlocuieşte host-ul cu un FQDN real şi adaugă o
# secţiune ‘tls:‘ cu un Secret de tip kubernetes.io/tls (cert-manager etc.).
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
metadata:
  name: epimodel-ingress
  namespace: epimodel
  labels:
    app.kubernetes.io/part-of: epimodel
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "64m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "120"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "120"
spec:
  ingressClassName: nginx
  rules:
    - host: epimodel.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 8080
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

Listing E.6: `70-ingress.yaml` — Ingress nginx pentru rutare HTTP.

Alternativă pentru dezvoltare locală

În paralel cu manifestele Kubernetes, repository-ul include și o configurație Docker Compose care reproduce aceeași topologie pe o singură mașină, fără nevoia unui cluster. Aceasta este utilă pentru dezvoltare locală și pentru verificări rapide ale fluxurilor end-to-end fără costul operațional al unui cluster K8s real.

```
services:
  yugabyte:
```

```
    image: yugabytedb/yugabyte:2.21.1.0-b271
    container_name: emedicine-yugabyte
    command:
      - bin/yugabyted
      - start
      - --background=false
      - --tserver_flags=ysql_num_shards_per_tserver=4
    ports:
      - "5433:5433"
      - "7000:7000"
      - "9000:9000"
    volumes:
      - yugabyte-data:/root/var
    healthcheck:
      test: ["CMD-SHELL", "bin/ysqlsh -h yugabyte -p 5433 -U
 yugabyte -c 'SELECT 1' >/dev/null 2>&1 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 30s

  redis:
    image: redis:7-alpine
    container_name: emedicine-redis
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  migrate:
    image: migrate/migrate:v4.17.1
    container_name: emedicine-migrate
    depends_on:
      yugabyte:
        condition: service_healthy
    volumes:
      - ./db/migrations:/migrations
    entrypoint:
```

```
      - migrate
      - -path=/migrations
      - -database=postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable
      - up
    restart: "no"

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    container_name: emedicine-api
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      HTTP_ADDR: ":8080"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
      REDIS_URL: "redis://redis:6379/0"
      JWT_SECRET: "${JWT_SECRET:-dev-secret-change-me}"
      RUNNER_DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    ports:
      - "8080:8080"
    restart: unless-stopped

  runner-python:
    build:
      context: ./runners/python
      dockerfile: Dockerfile
    container_name: emedicine-runner-python
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
```

```
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  runner-r:
    build:
      context: ./runners/r
      dockerfile: Dockerfile
    container_name: emedicine-runner-r
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  runner-julia:
    build:
      context: ./runners/julia
      dockerfile: Dockerfile
    container_name: emedicine-runner-julia
    depends_on:
      migrate:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    environment:
      REDIS_URL: "redis://redis:6379/0"
      DATABASE_URL: "postgres://yugabyte:yugabyte@yugabyte:5433/yugabyte?sslmode=disable"
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: emedicine-frontend
```

```
    depends_on:
      - api
    ports:
      - "5173:80"
    restart: unless-stopped

volumes:
  yugabyte-data:
```

Listing E.7: `docker-compose.yml` — Configurația echivalentă pentru mediul de dezvoltare local.

Anexa F

Documentația API REST

API-ul REST este implementat în pachetul `internal/api` al backend-ului Go. Rutele sunt definite centralizat în routerul `chi`, iar handler-ele sunt separate pe domenii: autentificare, dataseturi, simulări, benchmarkuri și health check. Specificația tabelară a endpointurilor se află în Table 4.2; în această anexă sunt incluse fișierele relevante pentru definirea rutelor și a formatului comun de răspuns.

```
package router

import (
    "net/http"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/chi/v5/middleware"
    "github.com/go-chi/cors"

    "github.com/tudorgeana/emedicine-api/internal/api/handlers"
    authmw "github.com/tudorgeana/emedicine-api/internal/api/middleware"
)

type Deps struct {
    Health      *handlers.HealthHandler
    Auth        *handlers.AuthHandler
    Datasets    *handlers.DatasetsHandler
    Simulations *handlers.SimulationsHandler
    Benchmarks  *handlers.BenchmarksHandler
    JWT         *authmw.JWTMiddleware
}

```

```
23 func New(deps Deps) http.Handler {
24     r := chi.NewRouter()
25 
26     r.Use(middleware.RequestID)
27     r.Use(middleware.RealIP)
28     r.Use(middleware.Logger)
29     r.Use(middleware.Recoverer)
30     r.Use(middleware.Timeout(60))
31     r.Use(cors.Handler(cors.Options{
32         AllowedOrigins:   []string{"*"},
33         AllowedMethods:   []string{"GET", "POST", "PUT", "
   DELETE", "OPTIONS"},
34         AllowedHeaders:   []string{"Accept", "Authorization",
   "Content-Type"},
35         AllowCredentials: false,
36         MaxAge:           300,
37     }))
38 
39     r.Route("/api", func(r chi.Router) {
40         r.Get("/health", deps.Health.Get)
41 
42         r.Route("/auth", func(r chi.Router) {
43             r.Post("/register", deps.Auth.Register)
44             r.Post("/login", deps.Auth.Login)
45         })
46 
47         r.Group(func(r chi.Router) {
48             r.Use(deps.JWT.Authenticate)
49 
50             r.Route("/datasets", func(r chi.Router) {
51                 r.Get("/", deps.Datasets.List)
52                 r.Post("/", deps.Datasets.Create)
53                 r.Get("/{id}", deps.Datasets.Get)
54                 r.Delete("/{id}", deps.Datasets.Delete)
55             })
56 
57             r.Route("/simulations", func(r chi.Router) {
58                 r.Get("/", deps.Simulations.List)
59                 r.Post("/", deps.Simulations.Create)
60                 r.Get("/{id}", deps.Simulations.Get)
61                 r.Get("/{id}/results", deps.Simulations.
```

```
    Results)
                r.Get("/{id}/export", deps.Simulations.Export)
            })

            r.Get("/benchmarks", deps.Benchmarks.Aggregate)
        })
    })

    return r
}
```

Listing F.1: Routerul REST al API-ului Go.

```
package handlers

import (
    "encoding/json"
    "net/http"
)

type envelope struct {
    Data  any          `json:"data,omitempty"`
    Error *errPayload `json:"error,omitempty"`
}

type errPayload struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
}

func writeJSON(w http.ResponseWriter, status int, data any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(envelope{Data: data})
}

func writeError(w http.ResponseWriter, status int, message
    string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(envelope{Error: &errPayload{
    Code: status, Message: message}})
```

```
28 }
29
30 func decodeJSON(r *http.Request, dst any) error {
31     dec := json.NewDecoder(r.Body)
32     dec.DisallowUnknownFields()
33     return dec.Decode(dst)
34 }
```

Listing F.2: Formatul comun al răspunsurilor JSON.